

第一章算法设计基础

算法设计的主要任务是描述问题的解决方案。算法是解决问题的方法，是对特定问题求解步骤的一种描述，是指令的有限序列。算法必须满足下列特性：有穷性；确定性；可行性。

算法的描述方法：

流程图。

表 1-1 流程图的基本符号		
图形符号	名称	含义
	起止框	开始或结束
	处理框	处理或运算
	输入/输出框	输入/输出
	判断框	根据判断结果,执行两条路径中的某一条路径
	控制流	执行的路径,箭头代表方向

伪代码。

例 1.2 用伪代码描述欧几里得算法。

解：欧几里得算法用于求两个自然数的最大公约数。设两个自然数是 m 和 n ，欧几里得算法的基本思想是将 m 和 n 辗转相除直到余数为 0。例如， $m=35, n=25$ ，计算过程如图 1-2 所示，当余数 r 为 0 时，除数 n 就是所求的最大公约数。算法如下。

	被除数 m	除数 n	余数 r
第 1 次相除	35	25	10
第 2 次相除	25	10	5
第 3 次相除	10	5	0

图 1-2 欧几里得算法的计算过程

算法：欧几里得算法 ComFactor

输入：两个自然数 m 和 n

输出： m 和 n 的最大公约数

- $r = m \bmod n$;
- 循环直到 r 等于 0：
 - $m = n$;
 - $n = r$;
 - $r = m \bmod n$;
- 输出 n ;

如何评价算法：

(1) 正确性：算法能满足具体问题的需求，即对于任何合法的输入，算法都会得出正确的结果。

(2) 健壮性：算法对非法输入的抵抗能力，即对于错误的输入，算法应能识别并作出处理，而不是产生错误动作或陷入瘫痪。

(3) 可理解性。

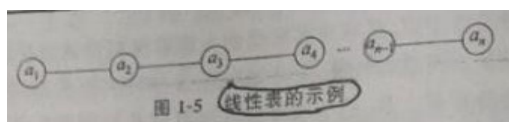
(4) 抽象分级：如果算法的求解步骤太多，可以将某些求解步骤合并为一个较抽象的处理，然后用其他算法描述这个较抽象的处理。

(5) 高效性：算法效率包括时间效率和空间效率，时间效率显示了算法运行得有多快；而空间效率则显示了算法需要多少额外的存储空间。

基本的数据结构数据结构：是指相互之间存在一定关系的数据元素的集合。利用计算机求解问题的第一步就是将实际问题抽象为合适的数据模型。

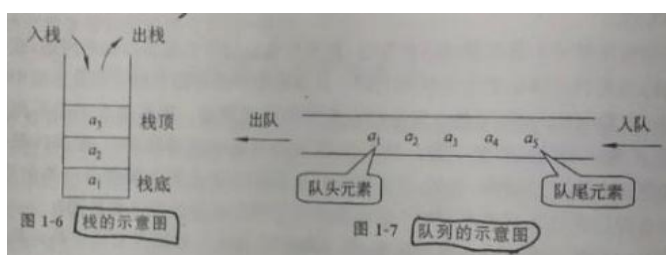
栈和队列是两种特殊的线性表。

线性表简称表，是 $n(n \geq 0)$ 个数据元素的有限序列。一个非空表通常记为： $L=(a_1, a_2, \dots, a_n)$ ，其中， L 表示一个线性表， $a_i(1 \leq i \leq n)$ 称为数据元素，下角标 i 表示该元素在线性表中的位置或序号。任意一对相邻的数据元素 a_{i-1} 和 $a_i(1 < i \leq n)$ 之间存在序偶关系 $\langle a_{i-1}, a_i \rangle$ ，且 a_{i-1} 称为 a_i 的前驱， a_i 称为 a_{i-1} 的后继。线性表的逻辑结构如图 1-5 所示。



栈:是限定仅在一端进行插入和删除操作的线性表，允许插入和删除的一端称为栈顶，另一端称为栈底。如图 1-6 所示，栈中有三个元素，插入（也称入栈、进栈）元素的顺序是 a_1 、 a_2 、 a_3 。当需要删除（也称出栈、弹栈）元素时只能删除 a_3 ，换言之，任何时刻出栈的元素都只能是栈顶元素，即最后入栈者最先出栈，所以栈中元素具有后进先出的特性。

队列:是只允许在一端进行插入操作，在另一端进行删除操作的线性表，允许插入（也称入队、进队）的一端称为队尾（queue tail），允许删除（也称出队）的一端称为队头。图 1-7 是一个有 5 个元素的队列，入队的顺序为 a_1 、 a_2 、 a_3 、 a_4 、 a_5 ，出队的顺序依然是 a_1 、 a_2 、 a_3 、 a_4 、 a_5 ，即最先入队者最先出队，所以队列中的元素具有先进先出的特性。

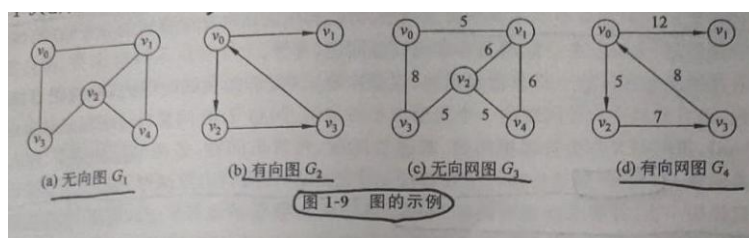


树：是 $n(n \geq 0)$ 个结点的有限集合。任意一棵非空树满足以下条件：(1)有且仅有一个特定的称为根的结点。(2)当 $n > 1$ 时，除根结点之外的其余结点被分成 $m(m > 0)$ 个互不相交的有限集合 $T_1, T_2, \dots, T_m$ ，其中每个集合又是一棵树，并称为这个根结点的子树。

图 1-8 是一棵具有 8 个结点的树， $T = (A, B, C, D, E, F, G, H)$ ，结点 A 为树 T 的根结点，除根结点 A 之外的其余结点分为两个不相交的集合： $T_1 = \{B, D, E, F\}$ 和 $T_2 = \{C, G, H\}$ ， T_1 和 T_2 构成了根结点 A 的两棵子树。

图：由 $n(n \geq 0)$ 个顶点的有限集合和顶点之间边的集合组成，通常表示为： $G = (V, E)$ ，其中，G 表示一个图，V 是顶点的集合，E 是顶点之间边的集合。例如，在图 1-9(a)所示 G_1 中，顶点集合 $V = \{v_0, v_1, v_2, v_3, v_4\}$ ，边的集合 $E = \{(v_0, v_1), (v_0, v_3), (v_1, v_2), (v_1, v_4), (v_2, v_3), (v_2, v_4)\}$ 。如果图中任意两个顶点之间的边都没有方向，则称该图为无向图，否则称该图为有向图。图 1-9(a)是个有向图，图 1-9(b)是一个有向图。

权：通常是对图中的边赋予的数值量。边上带权的图称为带权图或网图。图 1-9(c)是一个无向网图，图 1-9(d)是一个有向网图。



第二章 算法分析基础

算法分析：指的是对算法所需要的时间和空间进行估算。

输入规模与基本语句

算法的时间复杂度分析是一种事前分析估算的方法，是对算法所消耗资源的一种渐近分析方法。所谓渐近分析是指忽略具体机器、编程语言和编译器的影响，只关注在输入规模增大时算法运行时间的增长趋势。渐近分析降低了算法分析的难度，是从数量级的角度评价算法的效率。

撇开与计算机软硬件有关的因素，影响算法时间代价的最主要因素是输入规模。输入规模是指输入量的多少，一般来说，可以从问题描述中得到。

运行算法所需要的时间 T 是输入规模 n 的函数，记作 $T(n)$ 。

为了客观地反映一个算法的运行时间，可以用算法中基本语句的执行次数来度量算法的工作量。基本语句是执行次数与整个算法的执行次数成正比的语句，

例 2.1 对如下顺序查找算法，请找出输入规模和基本语句。

```
int SeqSearch(int A[],int n,int k) //在数组 A[n] 中查找值为 k  
的记录
```

```
{  
    for (int i=0; i<n; i++)  
        if (A[i] == k) break;
```

```

    if (i == n) return 0; //查找失败，返回失败的标志。

    else return (i+1); //查找成功，返回记录的序号。因为 i 从 0
开始。

}

```

解：算法的运行时间主要耗费在循环语句，循环的执行次数取决于待查找记录个数 n 和待查值 k 在数组中的位置，每执行一次 `for` 循环，都要执行一次元素比较操作。因此输入规模是待查找的记录个数，基本语句是比较操作 ($A[i]==k$)。

例 2.2 对如下起泡排序算法，请找出输入规模和基本语句。

```

void BubbleSort (int r[ ], int n)
{
    int j, temp, bound, exchange=n-1; //第一趟排序区间 [0,n-1]
    while (exchange!=0) //当上一趟排序有记录交换时
    {
        bound=exchange; exchange=0;
        for (j = 0; j < bound; j++) //一趟起泡排序区间是 [0,bound]
            if(r[j]>r[j+1])
            {
                temp=r[j]; r[j]=r[j+1];r[j+1]=temp;
                exchange=j; //记载每一次记录交换的位置
            }
    }
}

```

```

    }
}
}

```

解：算法由两层嵌套的循环组成，内层循环的执行次数取决于每一趟待排序区间的长度，也就是待排序记录个数，外层循环的终止条件是在一趟排序过程中没有交换记录的操作。是否有交换记录的操作取决于相邻两个元素的比较结果，也就是说，每执行一次 **for** 循环，都要执行一次比较操作，而交换记录的操作却不一定执行。因此输入规模是待排序的记录个数 n ，基本语句是比较操作 ($r[j]>r[j+1]$)。

例 2.3 如下算法实现将两个升序序列合并成一个升序序列，请找出输入规模和基本语句。

```

void Union(int A[],int n,int B[],intm,intC[]) //合并 A[n]和 B[m]
{
    int i=0,j=0,k=0;
    while(i<n&& j<m)
    {
        if (A[i] <=B[j])  C[k++]=A[i++]; //较小者存入 C[k]
        else C[k++]=B[j++];
    }
    while (i<n) C[k++]=A[i++]; //序列 A 中还有剩余记录
}

```

`while(j<m)C[k++]=B[j++];` //序列 B 中还有剩余记录

解：算法由三个并列的循环组成，三个循环将序列 A 和 B 扫描一遍，因此，输入规模是两个序列的长度 n 和 m 之和。第 1 个循环根据比较结果决定执行两个赋值语句中的哪一个，因此，可以将比较操作 ($A[i] \leq B[i]$) 作为基本语句，第 2 个循环的基本语句是赋值操作 ($C[k++] = A[i++]$)，第 3 个循环的基本语句是赋值操作 ($C[k++] = B[j++]$)。时间复杂度为 $O(m+n)$ 。

例 2.5 分析例 2.1 中顺序查找算法的时间复杂度。

解：顺序查找从第一个元素开始，依次比较每一个元素，直至找到 k ，算法结束。如果数组的第一个元素恰好就是 k ，算法只要比较一个元素就行了，这是最好情况，时间复杂度为 $O(1)$ ；如果数组的最后一个元素是 k ，算法就要比较 n 个元素，这是最坏情况，时间复杂度为 $O(n)$ ；如果在数组中查找不同的元素，假设数据是等概率分布 / 查找第 $i(1 \leq i < n)$ 个元素的时间复杂度为 $O(n)$ ，即平均要比较大约一半的元素，这是平均情况，时间复杂度和最坏情况同数量级。

分析例 2.2 中起泡排序算法的时间复杂度。

解：起泡排序算法的基本语句是比较操作，执行次数取决于排序的趟数。最好情况下，待排序记录序列为升序，算法只执行一趟，进行了 $n-1$ 次比较，时间复杂度为 $O(n)$ 。最坏情况下，待排序记录序列为降序，每趟排序在无序序列中只有一个最大的记录被交换到最终

位置，算法执行 $n-1$ 趟，第 $i(1 \leq i < n)$ 趟排序执行了 $n-i$ 次比较，则记录的比较次数为 $n(n-1)/2$ ，时间复杂度为 $O(n^2)$ 。

例 2.8 分析例 2.2 起泡排序算法的空间复杂度。

起泡排序算法的初始序列和排序结果都在数组 $r[n]$ 中，在排序算法的执行过程中设置了 3 个简单变量，其中，变量 `exchange` 记载每趟排序最后一次交换的位置，`bound` 表示每趟排序的待排序区间，变量 `temp` 作为交换记录的临时单元，因此，算法的空间复杂度为 $O(1)$ 。

如果算法所需的辅助空间相对于问题的输入规模来说是一个常数，称此算法为就地（或原地）工作。例如，起泡排序算法属于就地排序。

例 2.9 分析例 2.3 合并算法的空间复杂度。

解：在合并算法的执行过程中，可能会破坏原来的有序序列，因此，合并不能就地进行需要将合并结果存入另外一个数组。设序列 A 的长度为 n ，序列 B 的长度为 m ，则合并后有序序列的长度为 $n+m$ ，因此，算法的空间复杂度为 $O(n+m)$ 。

第五章蛮力法

蛮力法（也称穷举法或枚举法），是一种简单直接地解决问题的方法。

蛮力法的设计思想：蛮力法采用一定的策略依次处理待求解问题的所有元素，从而找出问题的解。

(1)理论上，蛮力法可以解决可计算领域的各种问题。

(2)蛮力法经常用来解决一些较小规模的问题。

(3)蛮力法可以作为某类问题时间性能的下界，来衡量同样问题的其他算法是否具有更高的效率。

一个简单的例子：百元买百鸡问题。

【问题】已知公鸡 5 元一只，母鸡 3 元一只，小鸡 1 元三只，用 100 元钱买 100 只鸡，问公鸡、母鸡、小鸡各多少只。

【想法】设公鸡、母鸡和小鸡的个数分别为 x 、 y 、 z ，则有如下方程组成立：

$$0 \leq x \leq 20 \quad 0 \leq y \leq 33 \quad 0 \leq z \leq 100$$

$$\text{且 } x+y+z=100 \quad 5x+3y+z/3=100$$

则百元买百鸡问题转换为求方程组的解。应用蛮力法求方程组的解只能依次试探变量 x 、 y 和 z 的值，验证 x 、 y 和 z 的某个特定值是否能够使方程组成立。

【算法实现】设变量 x 、 y 和 z 分别表示公鸡、母鸡和小鸡

的个数，由于方程组可能有多个解，设变量 `count` 表示解的个数，注意到小鸡 1 元三只，在判断总价是否满足方程时要先判断 `z` 是否是 3 的倍数。程序如下：

```
void Chicken()
{
    int x,y,z,count=0;
    for(x=0;x<=20;x++)
    {
        for (y = 0; y <= 33;y++)
        {
            z=100-x-y;    //满足共 100 只
            if((z%3==0)&&(5*x+3*y+z/3==100) //满足总价 100 元
            {
                count++; //解的个数加 1
                cout<<"公鸡: "<<x<<"母鸡: "<<y<<"小鸡: "<<z<<endl;
            }
        }
        if(count==0)
            cout<<"问题无解"<<endl;
    }
}
```

【算法分析】 算法由两层嵌套循环组成，基本语句是条件表达式 $((z\%3==0)\&\&(5 * x + 3 * y + z/3 == 100))$ ，执行次

数为 $21 \times 34 = 714$ 。一般地，假设共买 n 只鸡，基本语句的执行次数为 $O(n^2)$ 。

顺序查找：

在查找集合中依次查找值为 k 的元素。若查找成功，给出该元素在查找集合中的位置；若查找失败，则给出失败信息。

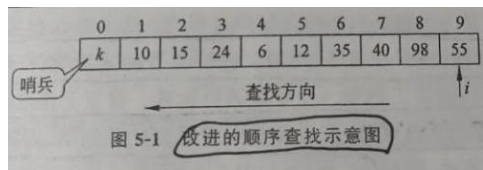
想法 1：与待查值进行比较。若相等，则查找成功，给出该元素在查找集合中的序号；若整个数组检测完仍未找到与待查值相等的元素，则查找失败，给出失败的标志 0。

算法实现 1：将查找集合存储在数组元素 $r[1] \sim r[n]$ 中，下标 i 初始化在数组的高端，这样查找结束时，若查找成功，下标 i 的值即元素的序号；若查找失败，下标 i 的值即为失败标志 0。注意，在查找过程中要检测比较位置是否越界，程序如下：

```
int SeqSearch1(int r[],int n,int k){  
    int i=n;  
    while (i > 0&& r[i]!=k)    //检测比较位置是否越界  
        i--;  
    return i;  
}
```

想法 2：为了避免在查找过程中每一次比较后都要判断查找位置是否越界，可以设置观察哨，即将待查值放在查找方向的“尽头”

处，则比较位置 i 至多移动到下标 0 处，也就是“哨兵”的位置。改进的查找过程如图 5-1 所示。



算法实现 2:

设函数 SeqSearch2 实现改进的顺序查找算法，程序如下：

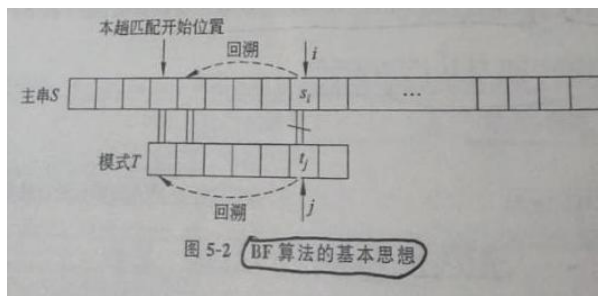
```
int SeqSearch2(int r[], int n, int k)
{
    int i=n;
    r[0]=k;    //设置哨兵
    while(r[i]!=k)    //不用检测比较位置是否越界
        i--;
    return i;
}
```

顺序查找算法和其改进算法的时间复杂度都是 $O(n)$ ，但是算法 SeqSearch1 要执行两个判断，而算法 SeqSearch2 只执行一个判断。实验表明，改进算法在待查找集合的长度大于 1000 时，进行一次顺序查找的时间几乎减少一半。

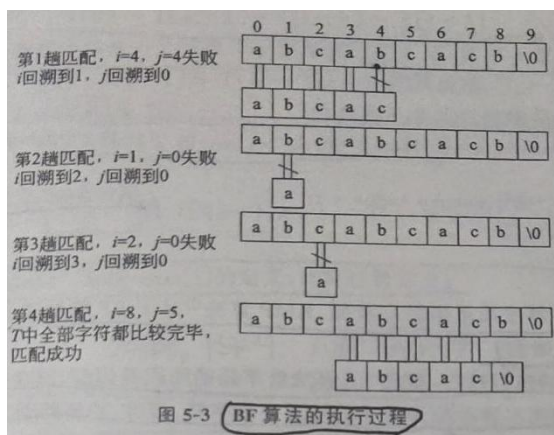
串匹配问题：

【问题】 给定两个字符串 S 和 T ，在主串 S 中查找子串 T 的过程称为串匹配，也称模式匹配， T 称为模式。

想法：应用蛮力法解决串匹配问题的过程是：从主串 S 的第一个字符开始和模式 T 的第一个字符进行比较，若相等，则继续比较二者的后续字符；若不相等，则从主串 S 的第二个字符开始和模式 T 的第一个字符进行比较。重复上述过程，若 T 中的字符全部比较完毕，则说明本趟匹配成功；若 S 中的字符全部比较完毕，则匹配失败。这个算法称为朴素的模式匹配算法，简称 BF 算法，如图 5-2 所示。



设主串 $S = \text{"abcabcacb"}$ ，模式 $T = \text{"abcac"}$ ，BF 算法的匹配过程如图 5-3 所示。



算法：设字符数组 S 存放主串，字符数组 T 存放模式，BF 算法如下。

算法：串匹配算法 BF

输入：主串 S ，模式 T

输出： T 在 S 中的位置

1. 初始化主串比较的开始位置 $index=0$;
2. 在串 S 和串 T 中设置比较的起始下标 $i=0, j=0$;
3. 重复下述操作，直到 S 或 T 的所有字符均比较完毕：
 - 3.1 如果 $S[i]$ 等于 $T[j]$ ，则继续比较 S 和 T 的下一对字符；
 - 3.2 否则，下一趟匹配的起始位置 $index++$ ，回溯下标 $i=index, j=0$;
4. 如果 T 中所有字符均比较完，则返回匹配开始位置的序号；

否则返回 0;

算法分析：设主串 S 长度为 n ，模式 T 长度为 m ，在匹配成功的情况下，考虑最坏情况，即每趟不成功的匹配都发生在串 T 的最后一个字符。时间复杂度为 $O(m*n)$ 。

算法实现：设字符数组 S 和 T 分别存储主串和模式，程序如下。

```
int BF(char S[ ], char T[ ])
{
    int index= 0, i=0,j=0;
    while ((S[i] != '\0') && (T[j] != '\0'))
    {
        if(S[i]== T[j]) {i++;j++;}
    }
}
```

```

else {index++; i= index; j=0; } //i 和 j 分别回溯

if (T[j]!='\0') return index+1; 返回本趟匹配开始位置的序号

else return 0;

}

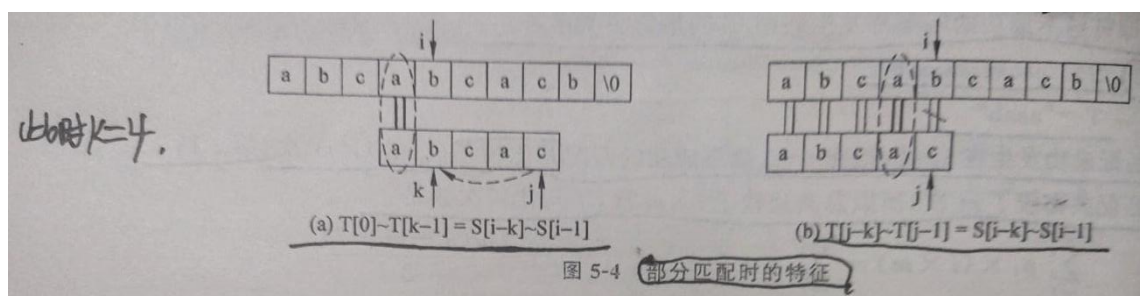
```

想法：BF 算法简单但效率较低，KMP 算法对于 BF 算法有了很大改进，基本思想是主串不进行回溯。

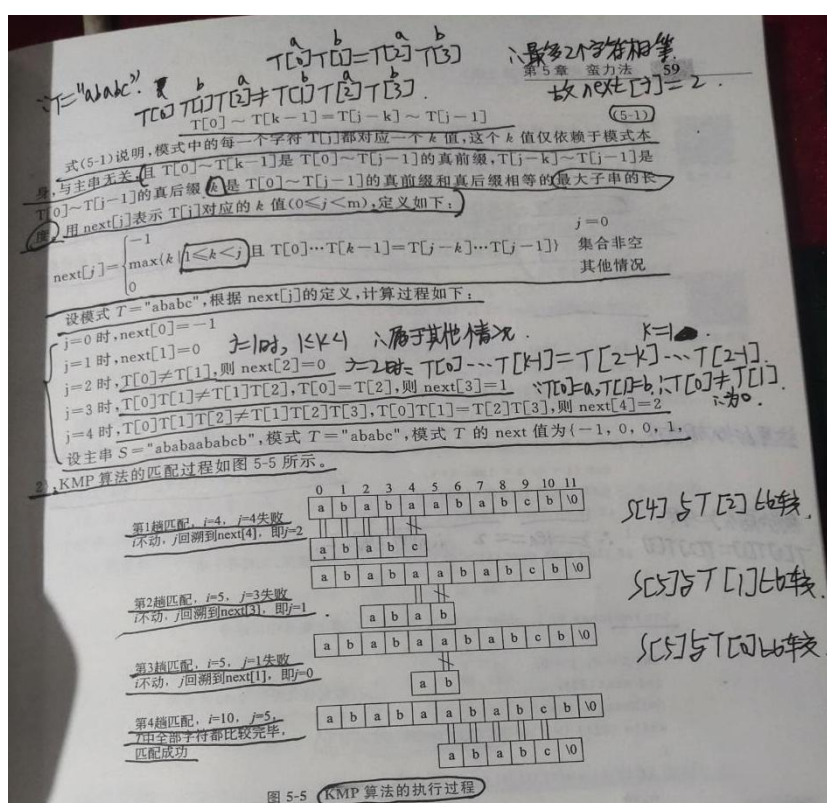
分析 BF 算法的执行过程，造成 BF 算法效率低的原因是回溯，即在某趟匹配失败后，对于主串 s 要回溯到本趟匹配开始字符的下一个字符，模式 T 要回溯到第一个字符，而这些回溯往往是不必要的。观察图 5-3 所示的匹配过程，在第 1 趟匹配过程中， $S[0] \sim S[3]$ 和 $T[0] \sim T[3]$ 匹配成功， $S[4] \neq T[4]$ 匹配失败。因为在第 1 趟中有 $S[1]=T[1]$ ，而 $T[0] \neq T[1]$ ，因此有 $T[0] \neq S[1]$ ，所以第 2 趟是不必要的，同理第 3 趟也是不必要的，可以直接到第 4 趟，进一步分析第 4 趟中的第一对字符 $S[3]$ 和 $T[0]$ 的比较是多余的，因为第 1 趟中已经比较了 $S[3]$ 和 $T[3]$ ，并且 $S[3]=T[3]$ ，而 $T[0]=T[3]$ ，因此必有 $S[3]=T[0]$ ，因此第 4 趟比较可以从第二对字符 $S[4]$ 和 $T[1]$ 开始进行，这就是说，第 1 趟匹配失败后，下标 i 不回溯，而是将下标 j 回溯至第 2 个字符，从 $T[1]$ 和 $S[4]$ 开始进行比较。

综上所述，希望某趟在 $S[i]$ 和 $T[j]$ 匹配失败后，下标 i 不回

溯，下标 j 回溯至某个位置 k ，从 $T[k]$ 和 $S[i]$ 开始进行比较。显然，关键问题是如何确定位置 k 。观察部分匹配成功时的特征，某趟在 $S[i]$ 和 $T[j]$ 匹配失败后，下一趟比较从 $S[i]$ 和 $T[k]$ 开始，则有 $T[0] \sim T[k-1] = S[i-k] \sim S[i-1]$ 成立，如图 5-4(a) 所示；在部分匹配成功时，有 $T[j-k] \sim T[j-1] = S[i-k] \sim S[i-1]$ 成立，如图 5-4(b) 所示。



由 $T[0] \sim T[k-1] = S[i-k] \sim S[i-1]$ 和 $T[j-k] \sim T[j-1] = S[i-k] \sim S[i-1]$ 可得：



【算法 2】在求得了模式 T 的 $next$ 值后，KMP 算法如下。

算法：串匹配算法 KMP

输入：主串 S ，模式 T

输出：T 在 S 中的位置

- 1.在串 S 和串 T 中分别设置比较的起始下标 $i=0, j=0$;
- 2.重复下述操作，直到 S 或 T 的所有字符均比较完毕;
 - 2.1 如果 $S[i]$ 等于 $T[j]$ ，则继续比较 S 和 T 的下一对字符;
 - 2.2 否则，将下标 j 回溯到 $next[j]$ 位置，即 $j=next[j]$;
 - 2.3 如果 j 等于 -1，则将下标 i 和 j 分别加 1，准备下一趟比较;
3. 如果 T 中所有字符均比较完毕，则返回本趟匹配开始位置的序号；
否则返回 0;

算法分析 2: 在求得模式 T 的 next 值后，KMP 算法只需将主串扫描一遍，设主串的长度为 n，则 KMP 算法的时间复杂度是 $O(n)$ 。

算法实现 2: 设函数 GetNext 用蛮力法求得模式 T 的 next 值，函数 KMP 实现 KMP 算法。

```
void GetNext (char T[ ], int next[])  
{  
    int i,j, len;  
    next[0]=-1;  
    for (j=1;T[j]!='\0';j++) //依次求 next[j]  
    {  
        for (len=j -1; len >= 1; len--) //相等子串的最大长度为 j-1  
            这里的 len 相当 k。  
        {  
            for (i = 0; i < len;i++) //比较 T[0]~T[len-1]与 T[j-len]~T[j-1]
```

```
if(T[i]!=T[j-len+i]) break;
```

```
if (i== len) {next[j]=len; break;} //例如前页 j=4 时, T[0]T[1]=T[2]T[3]
```

所以 i 等于 len 等于 2, 故 next[4]=2

```
if (len< 1) next[j]=0; //其他情况, 无相等子串
```

```
}
```

```
}
```

```
int KMP (char S[ ], char T[ ]) //求 T 在 S 中的序号
```

```
{
```

```
int i=0,j=0;
```

```
int next [80]; //假定模式最长为 80 个字符
```

```
GetNext(T,next);
```

```
while(S[i]!='\0'&&T[j]!='\0')
```

```
{
```

```
if(S[i]==T[j]) {i++;j++;}
```

```
else
```

```
{
```

```
j=next[j];
```

```
if(j==-1) {i++;j++;}
```

```
}
```

```
}
```

```
if (T[j] == '\0') return (i-j+1); // 返回本趟匹配开始位置序号 else
```

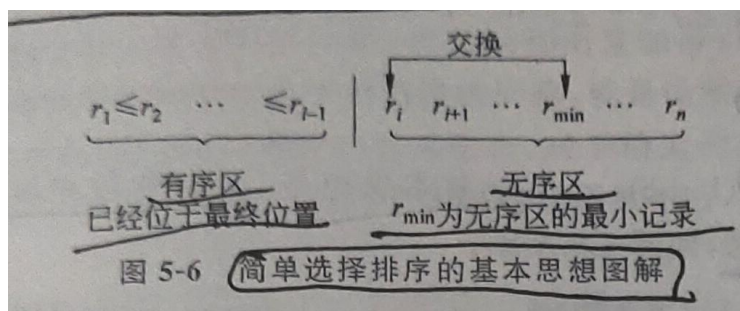
```
return 0;
```

}

排序问题中的蛮力法：

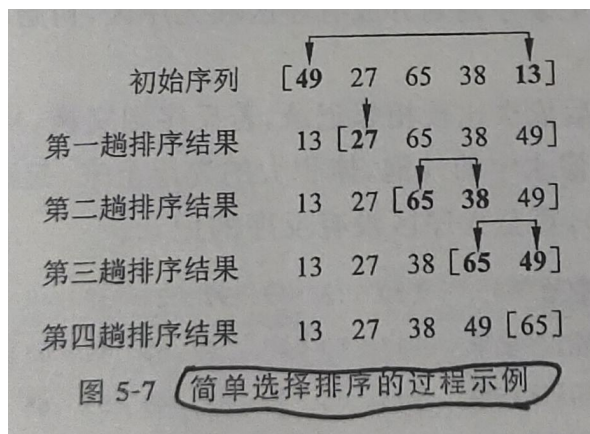
选择排序

【问题】选择排序的基本思想是：第 i 趟排序在无序序列 $r_i \sim r_n$ 中找到值最小的记录，并和第 i 个记录交换作为有序序列的第 i 个记录，如图 5-6 所示。



想法：图 5-7 给出了一个选择排序的例子（无序区用方括号括起来），具体的排序过程如下。

- (1) 将整个记录序列划分为有序区和无序区，初始时有序区为空，无序区含有待排序的所有记录。
- (2) 在无序区查找值最小的记录，将它与无序区的第一个记录交换，使得有序区扩展一个记录，同时无序区减少一个记录。
- (3) 重复执行步骤(2)，直至无序区只剩下一个记录。



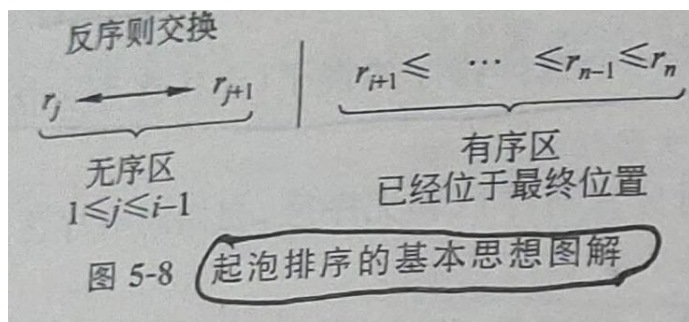
【算法实现】 设数组 $r[n]$ 存储待排序记录序列，注意，数组下标从 0 开始，则第 i 个记录存储在 $r[i-1]$ 中。程序如下：

```
void SelectSort(int r[ ],int n)
{
int i, j, index, temp;
for(i=0;i<n-1;i++)    //进行 n-1 趟选择排序
index=i;
for(j=i+1;j<n;j++)    //在无序区中查找最小记录
if (r[j]< r[index])    index=j;
temp=r[i]; r[i] =r[index]; r[index]=temp;    //交换记录
}
}
```

【算法分析】 算法 SelectSort 的基本语句是内层循环体的比较语句 ($r[i]<r[index]$)，执行次数为 $O(n^2)$ 。

起泡排序：

【问题】 基本思想是：两两比较相邻记录，如果反序则交换，直至没有反序的记录，如图 5.8 所示。



排序过程如下：

- (1) 将整个待排序的记录序列划分成有序区和无序区，初始时有序区为空，无序区包括所有待排序的记录。
- (2) 对无序区从前向后依次比较相邻记录，若反序则交换，从而使值较小的记录向前移，值较大的记录向后移（像水中的气泡，体积大的先浮上来，起泡排序因而得名）。
- (3) 重复执行步骤(2)，直至无序区没有反序的记录。

初始键值序列	[50 13 55 97 27 38 49 65]
第一趟排序结果	[13 50 55 27 38 49 65] 97
第二趟排序结果	[13 50 27 38 49] 55 65 97
第三趟排序结果	[13 27 38 49] 50 55 65 97
第四趟排序结果	13 27 38 49 50 55 65 97

图 5-9 起泡排序过程示例

【算法实现】注意，在一趟起泡排序过程中，如果有多个记录交换到最终位置，下一趟起泡排序将不再处理这些记录；另外，在一趟起泡排序过程中，如果没有交换记录操作则表明序列已经有序，算法将终止。起泡排序算法请参见 2.1.1 节。

【算法分析】参见 2.1.4 节。

分治法

分治法：

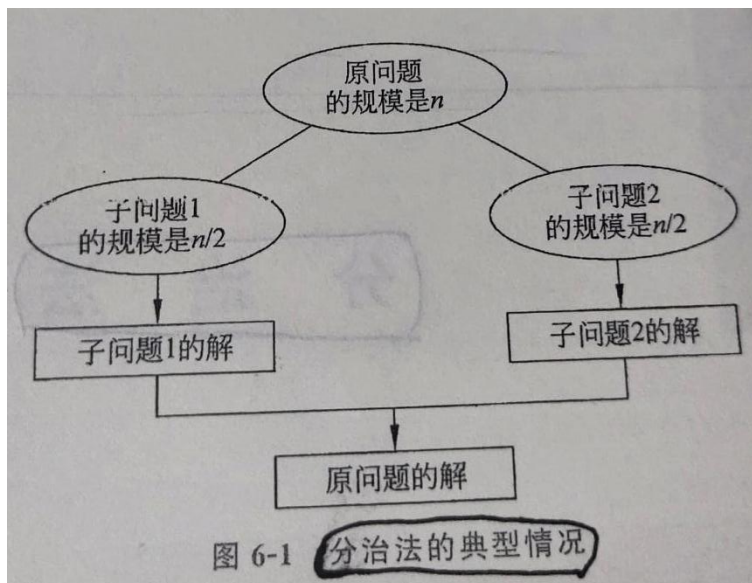
设计思想：分治法将一个难以直接解决的大问题划分成一些规模较小的子问题，分别求解各个子问题，再合并子问题的解得到原问题的解。一般来说，分治法的求解过程由以下三个阶段组成。

(1)划分：把规模为 n 的原问题划分为 k 个（通常 $k=2$ ）规模较小的子问题。

(2)求解子问题：分别求解各个子问题。各子问题的解法与原问题的解法通常是相同的，可以用递归的方法求解。

(3)合并：把各个子问题的解合并起来。

在进行问题划分时，子问题的规模最好大致相同，也就是将原问题划分成规模相等的 k 个子问题。另外，各子问题之间最好相互独立，这涉及分治法的效率，如果各子问题不是独立的，则分治法需要重复求解公共的子问题，此时虽然也可以用分治法，但一般用动态规划法较好。

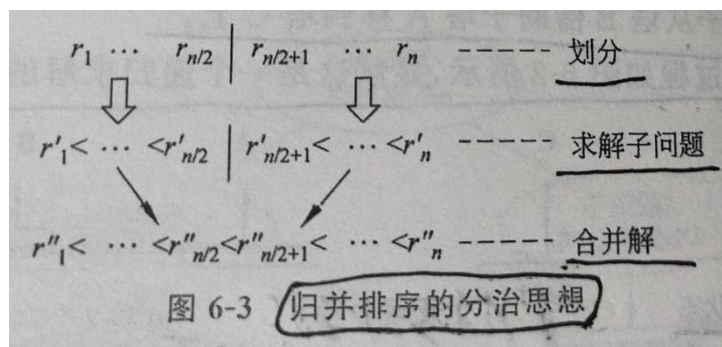


递归：是一种描述问题和解决问题的基本方法，递归程序直接调用自己或通过一系列调用语句间接调用自己，将待求解问题转化为解法相同的子问题，最终实现问题求解。具体地，递归方法的求解思想是：将一个难以直接解决的原问题分解为两部分。一部分是规模足够小的子问题，可以直接求解；另一部分是一些规模较小的子问题，子问题的解决方法与原问题的解决方法相同。如果子问题的规模仍然不够小，则再将子问题分解为规模更小的子问题，如此分解下去，直到子问题的规模足够小，可以直接求解为止。因此，递归必须具备以下两个基本要素，才能在有限次计算后得出结果：①递归出口。定递归到何时终止，即递归的结束条件。②递归体：确定递归的方式，即原问题是如何分解为子问题的。

排序问题中的分治法：归并排序

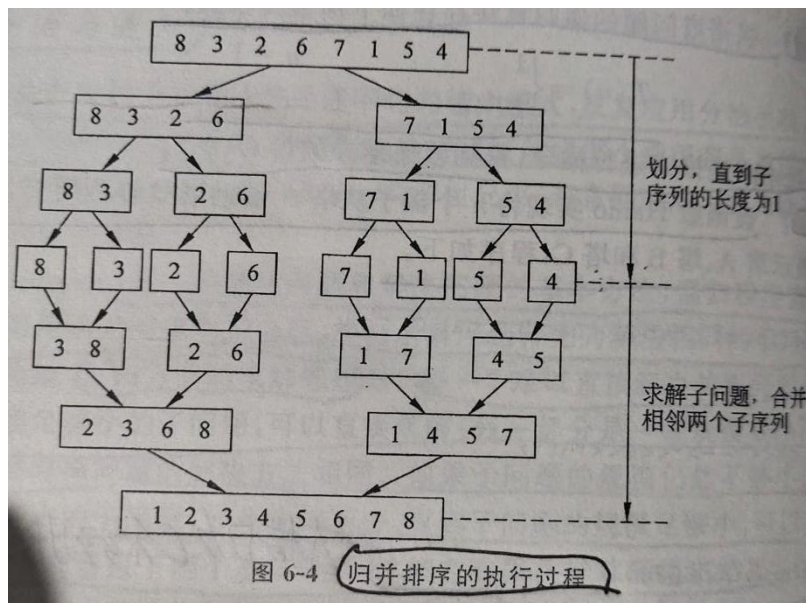
【问题】归并排序的分治策略如下(图 6-3)。

- (1)划分：将待排序序列从中间位置划分为两个长度相等的子序列。
- (2)求解子问题分别对这两个子序列进行排序，得到两个有序子序列。
- (3)合并：将这两个有序子序列合并成一个有序序列。



【想法】归并排序首先执行划分过程，将序列划分为两个子序列，如

果子序列的长度为 1，则划分结束，否则继续执行划分，结果将具有 n 个记录的待排序序列划分为 n 个长度为 1 的有序子序列；然后进行两两合并，得到 $n/2$ (向上取余)个长度为 2 (最后一个有序序列的长度可能是 1)的有序子序列，再进行两两合并，得到 $n/4$ (向上取余)个长度为 4 的有序序列(最后一个有序序列的长度可能小于 4)，以此类推，直至得到一个长度为 n 的有序序列。图 6-4 给出了归并排序的例子。



【算法】设对数组 $r[n]$ 进行升序排列，算法如下。

(伪代码)

算法：归并排序 MergeSort

输入：待排序数组 $r[n]$ ，待排序区间 $[s, t]$

输出：升序序列 $r[s] \sim r[t]$

1. 如果 s 等于 t ，则待排序区间只有一个记录，算法结束；
2. 计算划分的位置： $m = (s+t)/2$;
3. 对前半个子序列 $r[s] \sim r[m]$ 进行升序排列；

4. 对后半个子序列: $r[m+1] \sim r[t]$ 进行升序排列;

5. 合并两个升序序列: $r[s] \sim r[m]$ 和 $r[m+1] \sim r[t]$;

【算法分析】设待排序记录个数为 n , 则执行一趟合并算法的时间复杂度为 $O(n)$ 。归并排序的时间复杂度为 $O(n \log n)$ 。

算法实现: 归并排序的合并操作需要将两个相邻的有序子序列合并为一个有序序列, 在合并过程中可能会破坏原来的有序序列, 所以, 合并不能就地进行。设将有序子序列 $r[s] \sim r[m]$ 和 $r[m+1] \sim r[t]$ 合并为有序序列 $r1[s] \sim r1[t]$, 再将合并结果传回数组 $r[s] \sim r[t]$, 设函数 `Merge` 实现合并操作, 函数 `MergeSort` 实现归并排序程序如下。

```
void Merge(int r[], int s, int m, int t)
{
    //s 是高; m 是中间; t 是尾。

    int r1[t];

    int i=s, j=m+1, k=s;

    while(i<=m && j<=t)
    {
        if(r[i]<=r[j])    r1[k++]=r[i++];    //较小者放入 r1[k]
        else r1[k++]=r[j++];
    }

    while(i<=m)    //处理第一个子序列剩余记录
        r1[k++]=r[i++];
```

```

while(j<=t)    //处理第二个子序列剩余记录

    r1[k++]=r[j++];

for (i=s; i<=t;i++)    //将合并结果传回数组 r

    r[i]=r1[i];

}

void MergeSort(int r[],int s,int t)    //对序列 r[s]~r[t]进行归并排序

{

    if(s==t) return;    //只有一个记录，已经有序

    else{

int m=(s+t)/2;    //划分

MergeSort(r, s, m);    //归并排序前半个子序列

MergeSort(r, m+1, t);    //归并排序后半个子序列

Merge(r,s,m,t);    //合并两个有序子序列

    }

}

```

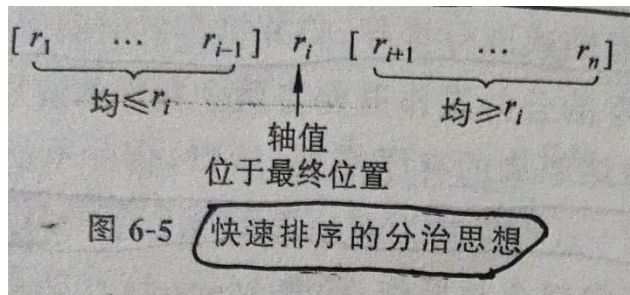
快速排序

【问题】快速排序的分治策略如下(图 6-5)。

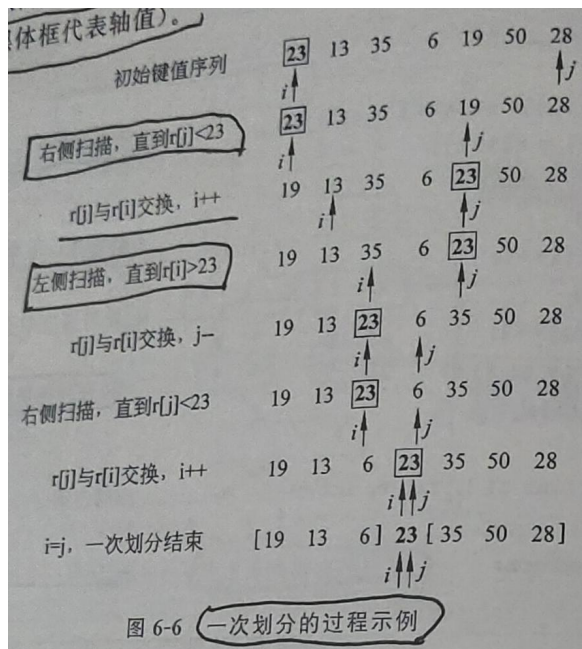
(1)划分：选定一个记录作为轴值，以轴值为基准将整个序列划分为两个子序列，轴值的位置在划分的过程中确定，并且左侧子序列的所有记录均小于或等于轴值，右侧子序列的所有记录均大于或等于轴值。

(2)求解子问题：分别对划分后的每一个子序列进行递归处理。

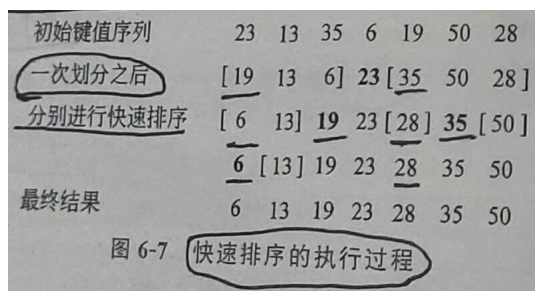
(3)合并：由于对子序列的排序是就地进行的，所以合并不需要执行任何操作。



【想法】首先对待排序记录序列进行划分，划分的轴值应该遵循平衡子问题的原则，使划分后两个子序列的长度尽量相等。轴值的选择有很多方法，例如，可以随机选出一个记录作为轴值，从而期望划分是平衡的。假设以第一个记录作为轴值，图 6-6 给出了一个划分的例子（黑体框代表轴值）。



以轴值为基准将待排序序列划分为两个子序列后, 对每一个子序列分别进行递归处理。图 6-7 所示是一个快速排序的完整的例子。



算法实现: 设函数 Partition 实现对序列 $r[\text{first}] \sim r[\text{end}]$ 进行划分, 函数 QuickSort 实现快速排序, 程序如下。

```
int Partition(int r[], int first, int end)
{
    int temp, i=first, j=end;

    while(i<j)
    {
        while (i<j && r[i] <= r[j])    j--; //右侧扫描
```

```

if(i<j)
{
temp=r[i]; r[i]=r[j]; r[j]=temp;    //将较小记录交换到前面
i++;
}

while(i<j&&[i]<=r[j]) i++;    //左侧扫描

if(i<j)
{
temp=r[i];r[i]=r[j];r[j]=temp;    //将较大记录交换到后面
j--;
}
}

return i;    //返回轴值记录的位置
}

void QuickSort(int r[], int first, int end)//快速排序
{
if (first<end)
{
int pivot = Partition(r, first, end);    //划分， pivot 是轴值的位置

QuickSort(r, first, pivot-1);    //对左侧子序列进行快速排序
QuickSort(r, pivot+1, end);    //对右侧子序列进行快速排序
}
}

```


}

算法分析：最好情况下，每次划分对一个记录定位后，该记录的左侧子序列与右侧子序列的长度相同。在具有 n 个记录的序列中，一次划分需要对整个待划分序列扫描一遍，所需时间为 $O(n)$ ，则 $O(n\log_2 n)$

最坏情况下，待排序记录序列正序或逆序，每次划分只得到一个比上一次划分少一个记录的子序列(另一个子序列为空)。此时，必须经过 $n-1$ 次递归调用才能把所有记录定位，而且第 i 趟划分需要经过 $n-i$ 次比较才能找到第 i 个记录的位置，因此，时间复杂度为： $O(n^2)$ 。

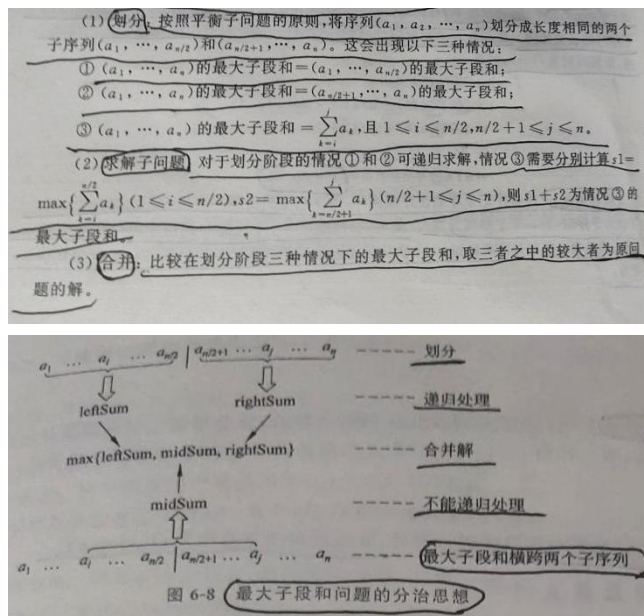
平均情况下，其数量级也为 $O(n\log_2 n)$ 。

由于快速排序是递归执行的，需要一个工作栈来存放每一层递归调用的必要信息，栈的最大容量与递归调用的深度一致。最好情况下要进行 $\lceil \log_2 n \rceil$ 次递归调用，栈的深度为 $O(\log_2 n)$ ；最坏情况下，要进行 $n-1$ 次递归调用，栈的深度为 $O(n)$ ；平均情况下，栈的深度为 $O(\log_2 n)$ 。

最大子段和问题：

问题：给定由 n 个整数（可能有负整数）组成的序列 $(a_1, a_2, \dots, a_n)$ ，最大子段和问题要求该序列子段之和的最大值。例如，序列 $(-20, 11, -4, 13, -5, -2)$ 的最大子段和为 20。($a_2 + a_3 + a_4 = 11 - 4 + 13 = 20$)

想法：最大子段和问题的分治策略如下(图 6.8)。



算法实现：最大子段和问题是按照位置进行划分的，设变量 `center` 表示序列的中间位置，数组 `a[n]` 存放整数序列，程序如下。

```
int MaxSum(int a[], int left, int right)
{
    int sum = 0, midSum = 0, leftSum = 0, rightSum = 0;
    int i, center, s1, s2, lefts, rights;
    if (left == right) //如果序列长度为 1，直接求解
        sum = a[left];
    else
```

```

{
center=(left + right)/2;      //划分
leftSum=MaxSum(a,left, center); //对应情况①，递归求解
rightSum = MaxSum(a, center+1, right); //对应情况②，递归求解
s1 = 0; lefts=0;    //以下对应情况③，先求解 s1
for (i = center; i>= left;i--)
{
    lefts+=a[i];
    if (lefts> s1)  s1= lefts;
}
s2=0;rights=0;    //再求解 s2
for (i= center + 1; i <= right; i++)
{
    rights+=a[i];
    if (rights > s2)  s2= rights;
}
midSum=s1+s2;      //计算情况③的最大子段和
if (midSum < leftSum) sum = leftSum; //合并解，取较大者
else sum= midSum;
if (sum < rightSum) sum= rightSum;
}
return sum;

```

}

算法时间复杂度： $O(n\log_2 n)$ 。

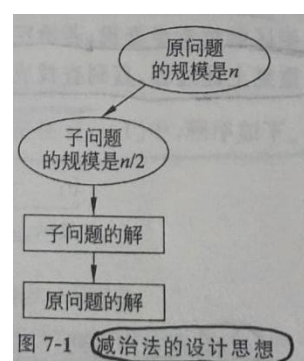
减治法：

分治法是把一个大问题划分为若干个子问题，分别求解各个子问题，然后再把子问题的解进行合并得到原问题的解。减治法同样也是把一个大问题划分为若干个子问题，但是只须求解其中的一个子问题，无须对子问题的解进行合并。所以，严格地说，减治法应该是一种退化了的分治法。

减治法的设计思想：减治法将原问题分解为若干个子问题，并且原问题(规模为 n)的解与子问题(规模通常为 $n/2$)的解之间存在某种确定的关系，这种关系通常表现为：

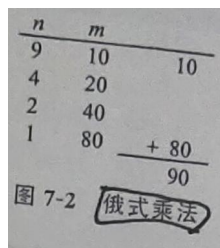
- (1)原问题的解只存在于其中一个较小规模的子问题中；
- (2)原问题的解与其中一个较小规模子问题的解之间存在某种对应关系。

由于原问题的解与子问题的解之间存在确定的关系，所以，只须求解其中一个较小规模的子问题就可以得到原问题的解。减治法的设计思想如图 7-1 所示。



一个简单的例子：俄式乘法

【问题】俄式乘法用来计算两个正整数 n 和 m 的乘积，运算规则是：如果 n 是偶数，计算 $n/2 \times 2m$ ；如果 n 是奇数，计算 $(n-1)/2 \times 2m + m$ ；当 n 等于 1 时，返回 m 的值。图 7-2 给出了利用俄式乘法计算 9×10 的例子。



n	m	
9	10	10
4	20	
2	40	
1	80	
		+ 80
		90

图 7-2 俄式乘法

【算法实现】表达式 $(n \& 1) == 0$ 判断整数 n 是否为偶数，表达式 $n >> 1$ 实现 $n/2$ 运算，表达式 $m << 1$ 实现 $2 \times m$ 运算，程序如下。

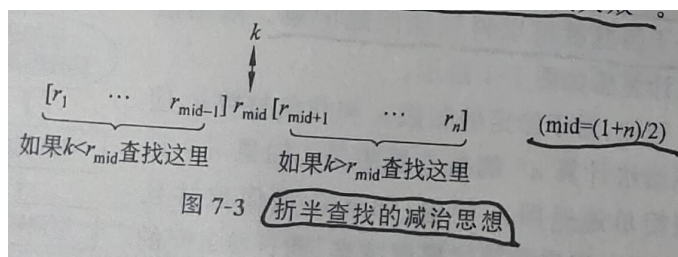
```
int RussMul(int n, int m)
{
    if (1 == n) return m;
    if ((n & 1) == 0) return RussMul(n >> 1, m << 1);
    else return RussMul(n - 1 >> 1, m << 1) + m;
}
```

查找问题中的减治法

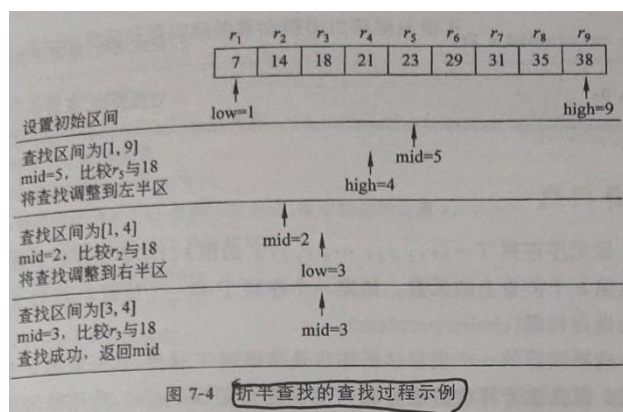
折半查找：

【问题】应用折半查找方法在一个有序序列中查找值为 k 的记录。若查找成功，返回记录 k 在序列中的位置；若查找失败，返回失败信息。

【想法】折半查找(图 7-3)利用了记录序列有序的特点，查找过程是：若给定值与中间记录相等，则查找成功；若给定值小于中间记录，则在记录序列的左半区继续折半查找；若给定值大于中间记录，则在记录序列的右半区继续折半查找。不断重复上述过程，直到查找成功，或所查找的区域无记录，则查找失败。



例如，在有序序列（7,14,18,21,23,29,31,35,38）中查找 18 的过程如图 7-4 所示。



【算法】折半查找算法用伪代码描述如下。

算法：折半查找 BinSearch

输入：有序序列（r1,r2,...,rn），待查值 k

输出：若查找成功，返回记录 k 的位置，若查找失败，返回失败标志

1. 设置初始查找区间：low=1;high=n;
2. 测试查找区间 [low,high] 是否存在，若不存在，则查找失败，返回 0;

3. 取中间点 $mid=(low+high)/2$; 比较 k 与 r_{mid} , 有以下三种情况:

3.1 若 $k < r_{mid}$, 则 $high=mid-1$; 查找在左半区进行, 转步骤 2;

3.2 若 $k > r_{mid}$, 则 $low=mid+1$; 查找在右半区进行, 转步骤 2;

3.3 若 $k=r_{mid}$, 则查找成功, 返回记录在表中位置 mid ;

【算法分析】折半查找将给定值与序列的中间记录进行比较, 每比较一次, 或者查找成功, 或者将查找区间减半, 所以时间复杂度为 $O(\log_2 n)$ 。

【算法实现】注意到数组下标从 0 开始, 则第 i 个元素存储在 $r[i-1]$ 中, 程序如下。

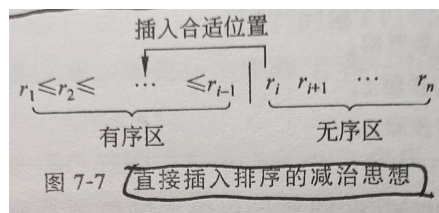
```
int BinSearch(int r[ ], int n, int k)
{
    int mid, low = 0, high=n-1;
    while (low<= high)    //当查找区间存在
    {
        mid=(low+ high) / 2;
        if (k < r[mid])    high = mid - 1;
        else if (k> r[mid])    low=mid+1;
        else return mid+1;//查找成功, 返回元素序号
    }
    return 0;
```


}

排序问题中的减治法

插入排序

【问题】插入排序的基本思想是：依次将待排序序列中的每一个记录插入一个已排好序的序列中，直至全部记录都排好序，如图 7-7 所示。



【想法】：插入排序属于减治法的减一技术，即每一趟排序后将问题规模减小 1。图 7-8 给出了一个直接插入排序的过程示例(方括号括起来的是无序区)，排序过程如下：

- (1)将整个待排序的记录序列划分成有序区和无序区，初始时有序区为待排序记录序列的第一个记录，无序区包括所有剩余待排序记录；
- (2)将无序区的第一个记录插入有序区的合适位置，从而使无序区减少一个记录，有序区增加一个记录；
- (3)重复执行步骤(2)，直至无序区没有记录。

初始序列	12	[15	9	20	10	6]
第一趟排序结果	12	15	[9	20	10	6]
第二趟排序结果	9	12	15	[20	10	6]
第三趟排序结果	9	12	15	20	[10	6]
第四趟排序结果	9	10	12	15	20	[6]
第五趟排序结果	6	9	10	12	15	20

图 7-8 直接插入排序的过程示例

【算法实现】：待排序记录序列存储在 $r[1] \sim r[n]$ 中，一般情况下，在 $i-1$ 个记录的有序区 $r[1] \sim r[i-1]$ 中插入一个记录 $r[i]$ 时，首先要查找 $r[i]$

的正确插入位置。最简单地，可以采用顺序查找。为了在寻找插入位置的过程中避免数组下标越界，在 $r[0]$ 处设置"哨兵"。在自 $i-1$ 起往前查找的过程中，同时后移记录。程序如下。

```
void InsertSort(int r[ ], int n)
{
    int i,j;
    for (i=2;i<=n;i++)    //从第 2 个记录开始执行插入操作
        //相当打方括号括起来的无序区
        {
            r[0]=r[i];    //暂存待插记录，设置哨兵。
            for (j=i-1; r[0] < r[j]; j--) //寻找插入位置
                r[j+1]=r[j];    //记录后移
            r[j+1]=r[0]; //因为该语句不再上面那个 for 循环中，执行了 for 语
                句的语句③： j--；    所以不是 r[j]=r[0];
        }
}
```

// $r[0]$ 的作用：一是进入查找插入位置的循环之前，暂存了 $r[i]$ 的值，使得不因记录的后移而丢失 $r[i]$ 的内容；二是在查找插入位置的循环中充当哨兵。

总结：时间复杂度 $O(n^2)$ 。

贪心法：

贪心法把一个复杂问题分解为一系列较为简单的局部最优选择，每一步选择都是对当前解的一个扩展，直至获得问题的完整解。贪心法的典型应用是求解最优化问题。

贪心法的设计思想：贪心法把一个复杂问题的求解过程划分为若干个阶段，每一个阶段都按照某种贪心选择策略进行选择，以迭代的方式求得每个阶段的当前最优解，每一次迭代都向给定目标前进一步，最终得到原问题的解。贪心法在解决问题的策略上目光短浅，只根据当前已有的信息作出贪心选择。

应用贪心法的关键是确定贪心选择策略，这种贪心选择策略只是根据当前信息作出最好的选择，不去考虑在后面看来这种选择是否合理。贪心法并不是从整体最优考虑，每一个阶段作出的选择都只是某种意义上的局部最优，这种局部最优选择并不总能获得整体最优解，但通常能获得近似最优解。如果一个问题的最优解只能通过蛮力法穷举得到，则贪心法不失为寻找问题近似最优解的一个较好办法。

一个简单的例子：付款问题

【问题】假设有面值为 5 元、2 元、1 元、5 角、2 角、1 角的纸币，需要找给顾客 4 元 6 角现金，付款问题要求找到一个付款方案，使得付出的纸币张数最少。

【想法】付款问题的贪心选择策略是在不超过应付款金额的前提下，选择面值最大的纸币。首先选出 1 张不超过 4 元 6 角的最大面值的纸币，即 2 元；其次选出 1 张不超过 2 元 6 角的最大面值的纸币，

即 2 元；再次选出 1 张不超过 6 角的最大面值的纸币，即 5 角，最后选出 1 张不超过 1 角的最大面值的纸币，即 1 角。总共付出 4 张纸币。

对于上述想法，付款问题应用贪心法得到的是整体最优解，但是如果把面值改为 3 元、1 元、8 角、5 角、1 角，需要付款 4 元 6 角现金时，则结果是 1 个 3 元、1 个 1 元、1 个 5 角和 1 个 1 角共 4 张纸币，但最优解却是 3 张纸币：1 个 3 元和 2 个 8 角。

【算法实现】 设数组 `money[6]` 存储纸币面值，为避免进行实数运算，将纸币的面值扩大 10 倍。程序首先将应付款 `sum` 扩大 10 倍变成整数，然后按面值从大到小依次试探，最后返回应付纸币的张数。程序如下。

```
int PayMoney(double sum)
{
    int money[6] = {50, 20, 10, 5, 2, 1};
    sum 相当于 4 元 6 角。
    int i, count=0, n=sum*10;
    while(n>0)
    {
        for(i = 0; i < 6; i++) /* 选取不超过 sum 的最大面值依次试探 */
        {
            if (n >= money[i])
            {
                count++;
                n -= money[i];
            }
        }
    }
}
```

```

cout<<"面值为"<<(double) money[i]/10<<endl;

n=n-money[i];

break;

    }

}

}

return count;

}

```

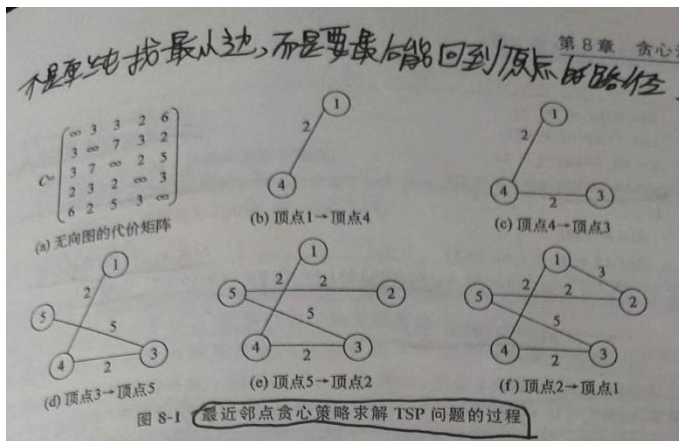
图问题中的贪心法

TSP 问题:

【问题】 TSP 问题是指旅行家要旅行 n 个城市，要求经历各个城市且仅经历一次然后回到出发城市，并要求所走的路程最短。

【想法】 TSP 问题的贪心策略可以采用最近邻点策略：从任意城市出发，每次在没有到过的城市中选择最近的一个，直至经过了所有城市，最后回到出发城市。如图 8-1(a)所示是一个无向图的代价矩阵，从顶点 1 出发，按照最近邻点的贪心策略，得到的路径是 $1 \rightarrow 4 \rightarrow 3 \rightarrow 5 \rightarrow 2 \rightarrow 1$ ，总代价是 14，求解过程如图 8-1(b)~(f)所示。

【算法】 设 c_{ij} 表示顶点 i 到顶点 j 的代价 ($1 \leq i, j \leq n$)，集合 V 存储图的顶点，集合 P 存储经过的边，出发顶点是 w ，算法如下。



算法：最近邻点贪心选择策略求解 TSP 问题

输入：无向带权图 $G=(V,E)$ ，出发顶点 w

输出：回路长度 TSPLength

1. 初始化： $P=\{\}$; $TSPLength=0$;
2. $u=w$; $V=V-\{w\}$;
3. 循环直到集合 P 包含 $n-1$ 条边：
 - 3.1 查找与顶点 u 邻接的最小代价边 (u,v) 并且 v 属于 V ;
 - 3.2 $P=P+\{(u,v)\}$; $V=V-\{v\}$; $TSPLength=TSPLength+c_{uv}$;
 - 3.3 $u=v$ ，转步骤 3 继续求解；
4. 输出 $TSPLength+C_{uv}$;

【算法分析】 算法共进行 $n-1$ 次贪心选择，每一次选择都需要查找满足贪心条件的最短边，因此时间复杂度为 $O(n^2)$ 。

需要说明的是，用最近邻点贪心策略求解 TSP 问题所得的结果不一定是最优解，例如，图 8-1(a) 中从顶点 1 出发的最优解是 $1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 1$ ，总代价只有 13。当图中顶点个数较多并且各边的

代价分布比较均匀时，最近邻点贪心策略可以给出较好的近似解，不过，这个近似解以何种程度近似于最优解，却难以保证。例如，在图 8-1(a)中，如果增大边(2,1)的代价，则总代价只好随之增加，没有选择的余地。

【算法实现】设函数 TSP 实现最近邻点贪心策略，数组 arc[n][n]存储图中各边的代价，设变量 TSPLength 存储回路长度，edgeCount 存储集合 P 中边的个数，标志数组 flag[n]表示某顶点是否在路径中，程序如下。

```
int TSP(int arc[100][100],int n,int w)
{
    int edgeCount = 0, TSPLength=0;
    int min, u, v, j;
    int flag[n]={0}; //顶点均未加入哈密顿回路
    u= w; flag[w]=1;
    while(edgeCount<n-1)    //循环直到边数等于 n-1
    {
        min=100;          //求 arc[u]中的最小值
        for (j =0; j < n; j++)
            if ((flag[j]==0) &&(arc[u][j]!=0)&&(arc[u][j]<min))
            {
                v=j;min=arc[u][j];
            }
        edgeCount++;
        TSPLength+=min;
        flag[v]=1;
        u=v;
    }
    return TSPLength;
}
```

```

}

TSPLength += min;    //将顶点加入路径

flag[v]=1; edgeCount++;

cout<<u<<"-->"<<v<<endl;    //输出经过的路径

u=v;    //下一次从顶点 v 出发

}

cout<<u<<"-->"<<w<<endl;    //输出最后的回边

return TSPLength + arc[u][w]; //返回回路长度

}

```

图着色问题

【问题】: 给定无向连通图 $G=(V,E)$, 图着色问题求图 G 的最小色数 k , 使得用 k 种颜色对 G 中的顶点着色, 可使任意两个相邻顶点着不同颜色。例如, 图 8-2 所示的无向图可以只用两种颜色着色, 将顶点 1、3 和 4 着一种颜色, 将顶点 2 和 5 着另外一种颜色。

【想法】 简单起见, 假定 k 个颜色的集合为 $(1,2,\dots,k)$ 。一种显然的贪心策略是选择一种颜色, 用该颜色为尽可能多的顶点着色。具体地, 取颜色 1, 依次考查图中未被着色的每个顶点, 如果该顶点的邻接点都未被着色, 则用颜色 1 为该顶点着色; 再取颜色 2, 依次考查图中的未被着色的每个顶点, 如果某顶点着颜色 2 与其相邻顶点的着色不发生冲突, 则用颜色 2 为该顶点着色; 如果还有未着色的顶点, 则取

颜色 3 并为尽可能多的顶点着色，依次类推。

【算法】设数组 `color[n]` 表示顶点的着色情况，算法如下。

算法：贪心法求解图着色问题 `ColorGraph`

输入：无向连通图 $G=(V,E)$

输出：最小色数 k

1. 所有顶点置未着色状态；颜色 k 初始化为 0;

2. 循环直到所有顶点均着色

2.1 取下一种颜色 $k++$;

2.2 循环变量 i 从 1 至 n 依次考查所有顶点：

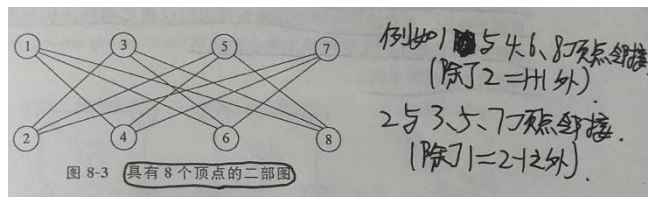
2.2.1 若顶点 i 已着色，则转步骤 2.2，考查下一个顶点；

2.2.2 若顶点 i 着颜色 k 不冲突，则 `color[i]=k`;

3. 输出各顶点的着色；

【算法分析】算法 `ColorGraph` 需要试探 k 种颜色，每种颜色都需要对所有顶点进行冲突测试，设无向图有 n 个顶点，则算法的时间复杂度是 $O(k*n)$ 。

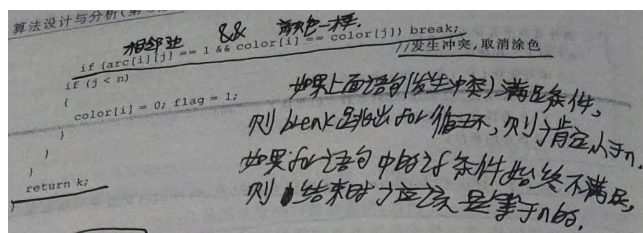
需要说明的是，贪心法求解图着色问题得到的不一定是最优解。考虑一个具有 $2n$ 个顶点的无向图，顶点的编号从 1 到 $2n$ ，当 i 是奇数时，顶点 i 与除了顶点 $i+1$ 之外的其他所有编号为偶数的顶点邻接，当 i 是偶数时，顶点 i 与除了顶点 $i-1$ 之外的其他所有编号为奇数的顶点邻接，这样的图称为二部图。图 8-3 所示是一个具有 8 个顶点的二部图。



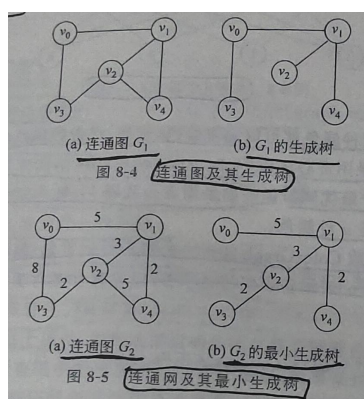
显然，二部图只用两种颜色就可以完成着色，例如，将奇数顶点全部着颜色 1，将偶数顶点全部着颜色 2。如果用贪心法以 1,3,...,2n-1, 2,4,...,2n 的顺序为二部图着色，则算法可以得到这个最优解。但是如果用贪心法以 1,2,3,...,2n 的顺序为二部图着色，则算法得到的解是 n 种颜色。

【算法实现】设函数 ColorGraph 实现贪心法进行图着色，数组 arc[n][n] 存储图中各边的代价，数组 color[n] 表示各顶点的着色情况，变量 flag 表示图中是否有尚未涂色的顶点，程序如下。

```
int ColorGraph(int arc[100][100], int n, int color[])
{
    int i, j, k=0, flag=1;
    while(flag==1)
    {
        k++; flag=0;    //取下一种颜色
        for(i=0; i<n; i++)
        {
            if (color[i] != 0) continue;    //顶点 i 已着色
            color[i]=k;    //顶点 i 着颜色 k
            for(j=0; j<n; j++)
```



最小生成树：在无向图中，若任意两个顶点之间均有路径，则称该图是连通图。连通图的生成树是包含图中全部顶点的一个极小连通子图，一棵具有 n 个顶点的生成树有且仅有 $n-1$ 条边，图 8-4 给出了连通图的生成树示例，显然，生成树可能不唯一。无向连通图的生成树上各边的权值之和称为生成树的代价。在图的所有生成树中，代价最小的生成树称为最小生成树。图 8-5 给出了连通网的最小生成树示例。

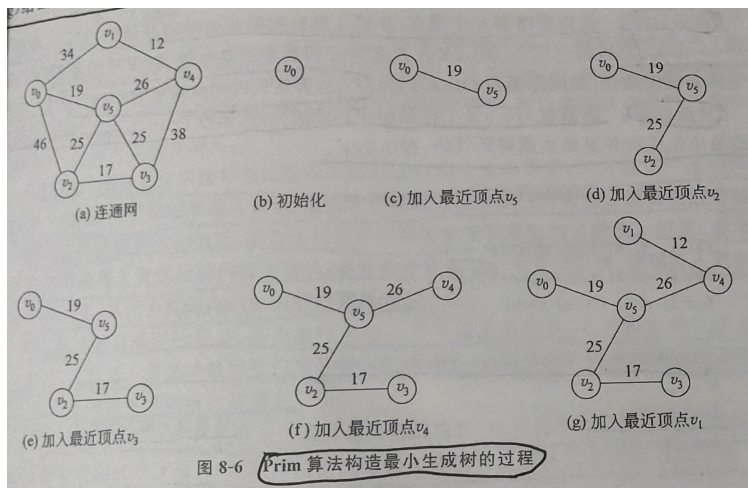


【问题】设 $G=(V,E)$ 是一个无向连通网，求 G 的最小生成树。

【想法】最小生成树问题的贪心策略可以采用最近顶点策略：任选一个顶点，并以此建立算生成树的根节点，每一步的贪心选择都把不在生成树的最近顶点添加到生成树中。prim 算法就应用了这个贪心策略，使生成树以一种自然的方式生长，即从任意顶点开始，每一步都为这

棵树添加一个分枝，直到生成树中包含全部顶点。

设最小生成树 $T=(U,TE)$ ，初始时 $U=P\{u_0\}$ (u_0 为任意顶点)， $TE=\{\}$ 。显然，Prim 算法的关键是如何找到连接 U 和 $V-U$ 的最短边来扩充生成树 T 。对于每一个不在当前生成树中的顶点 v 属于 $V-U$ ，都必须知道顶点 v 连接生成树的最短边信息。所以，对顶点 v 需要保存两个信息： $lowcost[v]$ 表示顶点 v 到生成树中所有顶点的最短边； $adjvex[v]$ 表示该最短边在生成树中的顶点。例如，对于图 8-6(a) 所示连通网，图 8-6(b)~(g) 给出了从顶点 v_0 出发，用 Prim 算法构造最小生成树的过程。

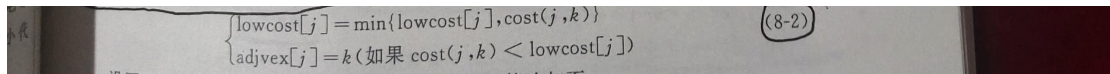


【算法】设数组 $adjvex[n]$ 和 $lowcost[n]$ 分别存储候选最短边的邻接点和权值，式(8-1)表示候选最短边 (v_i, v_k) 的权值为 w ，其中 v_i 属于 $V-U$, v_k 属于 U 。

$$\begin{cases} adjvex[i] = k \\ lowcost[i] = w \end{cases} \quad (8-1)$$

假设从顶点 w 出发构造最小生成树，初始时， $U=\{w\}$, $lowcost[w]=0$ ，表示顶点 w 已加入集合 U ，令 $adjvex[i]=0$, $lowcost[i]=(w,i)$ 的权值 ($1 \leq i \leq n-1$)。然后在数组 $lowcost[n]$ 中不断选取最小权值 $lowcost[k]$ ，并将 $lowcost[k]$ 置为 0，表示顶点 k 已加入集合 U 。由于顶点 k 从集合

V-U 进入集合 U 后, 候选最短边集发生了变化, 依据式(8-2)更新数组 $\text{adjvex}[n]$ 和 $\text{lowcost}[n]$ 。


$$\begin{cases} \text{lowcost}[j] = \min\{\text{lowcost}[j], \text{cost}(j, k)\} \\ \text{adjvex}[j] = k \text{ (如果 } \text{cost}(j, k) < \text{lowcost}[j] \text{)} \end{cases} \quad (8-2)$$

设图 G 采用代价矩阵存储, 起始顶点是 w, Prim 算法如下。

算法: Prim 算法

输入: 无向连通网 $G=(V, E)$ 起始点 w

输出: 最小生成树

1. 根据式(8-1)初始化数组 $\text{adjvex}[n]$ 和 $\text{lowcost}[n]$:

2. $U=\{w\}$; 输出顶点 w:

3. 重复执行下列操作 n-1 次;

3.1 在 lowcost 中选取最短边, 取 adjvex 中对应的顶点序号 k;

3.2 输出顶点 k 和对应的权值;

3.3 $U=U+\{k\}$;

3.4 根据式(8-2)调整数组 $\text{adjvex}[n]$ 和 $\text{lowcost}[n]$:

【算法分析】Prim 算法的时间复杂度为 $O(n^2)$ 。

【算法实现】设数组 $\text{arc}[n][n]$ 存储图中各边的代价, 函数 Prim 实现从顶点 w 开始构造最小生成树并返回生成树的代价, 程序如下。

```
int Prim(int arc[100][100], int n, int w)
{
    int i, j, k, min, minDist= 0;

    int lowcost[n], adjvex[n];
```

```

for (i = 0; i < n; i++)    //初始化
{
    lowcost[i]=arc[w][i]; adjvex[i] = w;
}

lowcost[w]=0;    //将顶点 w 加入集合 U

for(i=0;i<n-1;i++)
{
    min=100;    //假设权值均小于 100

    for(j=0; j < n; j++)    //寻找最短边的邻接点 k
    {
        if((lowcost[j]!=0)&&(lowcost[j]<min))
        {
            min=lowcost[j]; k=j;
        }
    }

    cout<<adjvex[k]<<"--"<<k<<endl;    //输出最小生成树的边

    minDist=minDist + lowcost[k];

    lowcost[k]=0;    //将顶点 k 加入集合 U 中

    for (j=0;j < n; j++)    //调整数组

    {

        if (arc[k][j] < lowcost[j])

```



```

{
    lowcost[j]=arc[k][j];
    adjvex[j]=k;
}
}
}

return minDist;
}

```

组合问题中的贪心法

背包问题：

【问题】给定 n 个物品和一个容量为 C 的背包，物品 $i(1 \leq i \leq n)$ 的重量是 w_i ，其价值为 v_i ，背包问题是如何选择装入背包的物品，使得装入背包中物品总价值最大。

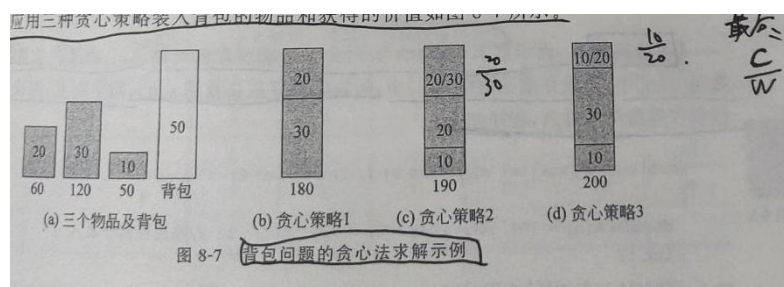
【想法】用贪心法求解背包问题的关键是如何选定贪心策略，使得按照一定的顺序选择每个物品，并尽可能地装入背包，直至背包装满。至少有三种看似合理的贪心策略：

(1)选择价值最大的物品，因为这可以尽可能快地增加背包的总价值。但是，虽然每一步选择都获得了背包价值的极大增长，但背包容量却可能消耗得太快，使得装入背包的物品个数减少，从而不能保证目标函数达到最大。

(2)选择重量最轻的物品，因为这可以装入尽可能多的物品，从而增加背包的总价值。但是，虽然每一步选择都使背包的容量消耗得慢了，但背包的价值却没能保证迅速增长，从而不能保证目标函数达到最大。

(3)以上两种贪心策略或者只考虑背包价值的增长，或者只考虑背包容量的消耗，为了求得背包问题的最优解，需要在背包价值增长和背包容量消耗二者之间寻找平衡。正确的贪心策略是选择单位重量价值最大的物品。

例如有 3 个物品，重量分别是(20,30,10)，价值分别为(60,120,50)，背包的容量为 50，应用三种贪心策略装入背包的物品和获得的价值如图 8-7 所示。



【算法】设背包容量为 C ，物品重量为 $w=(w_1, w_2, \dots, w_n)$ ，价值为 $v=(v_1, v_2, \dots, v_n)$ ，解为 $(x_1, x_2, \dots, x_n)$ ，其中 $x_i=0 (1 \leq i \leq n)$ 表示物品 i 没有被装入背包，算法如下：

算法：贪心法求解背包问题 KnapSack

输入：背包容量 C ，物品重量 $(w_1, w_2, \dots, w_n)$ ，价值 $(v_1, v_2, \dots, v_n)$

输出： $(x_1, x_2, \dots, x_n)$

1. 改变序列 w 和 v 的排列顺序，使其按单位重量价值降序排列；

2. 将 $x_i (1 \leq i \leq n)$ 初始化为 0；

3.考虑单位重量价值最大的物品： $i=1$;

4.循环直到 $w_i > C$:

4.1 将第 i 个物品放入背包： $x_i=1$;

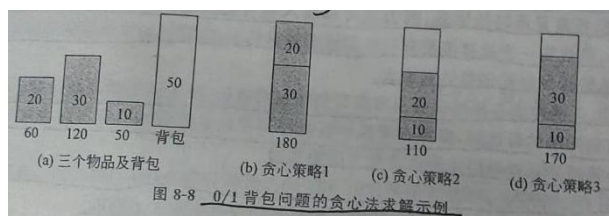
4.2 $C=C-w_i$;

4.3 $i++$;

5.最后装入的物品： $x_i=C/w_i$;

【算法分析】 算法 KnapSack 的时间主要消耗在将所有物品依其单位重量的价值从大到小排序，因此时间复杂度为 $O(n\log 2n)$ 。

背包问题可以用贪心法求解，而 0/1 背包问题却不能用贪心法求解。图 8-8 给出了一个用贪心法求解 0/1 背包问题的示例，可以看出，对于 0/1 背包问题，用贪心法之所以不能得到最优解，是由于物品不允许分割，因此，无法保证最终能将背包装满，部分闲置的背包容量降低了背包的单位重量价值。事实上，在考虑 0/1 背包问题时，应比较选择该物品和不选择该物品所导致的方案，然后再作出最优选择，由此产生许多互相重叠的子问题，所以，0/1 背包问题适合用动态规划法求解。



【算法实现】 设函数 KnapSack 实现贪心法求解背包问题， n 个物品的重量存放在数组 $w[n]$ 中，价值存放在数组 $v[n]$ 中，问题的解存放在

数组 $x[n]$ ，简单起见，假设物品已按单位重量降序排列，程序如下。

```
double KnapSack(int w[ ],int v[],int n,int C)
{
    double x[n]={0}, maxValue=0; //物品可部分装入
    int i;
    for (i= 0;w[i]<C; i++)
    {
        x[i]=1; //物品 i 装入一部分
        maxValue+=v[i]; //返回背包获得的价值
        C=C-w[i]; //背包剩余容量
    }
    x[i]=(double)C/w[i]; //物品 i 装入一部分
    maxValue+=x[i]*v[i];
    return maxValue; //返回背包获得的价值
}
```

活动安排问题

【问题】设有 n 个活动的集合 $E=\{1,2,\dots,n\}$ ，其中每个活动都要求使用同一资源(如演讲会场)，而在同一时间只有一个活动能使用这个资源。每个活动 $i(1\leq i\leq n)$ 都有一个要求使用该资源的起始时间 s_i 和一个结束时间 f_i 且 $s_i < f_i$ 。如果选择了活动 i ，则它在半开时间区间 $[s_i, f_i)$ 内

占用资源。若区间 $[s_i, f_i)$ 与区间 $[s_j, f_j)$ 不相交，则称活动 i 与活动 j 是相容的，活动安排问题要求在所给的活动集合中选出个数最多的相容活动。

【想法】贪心法求解活动安排问题的关键是如何选择贪心策略，使得按照一定的顺序选择相容活动，并能够安排尽量多的活动。至少有两种看似合理的贪心策略：

- (1)最早开始时间：这样可以增大资源的利用率。
- (2)最早结束时间：这样可以使下一个活动尽早开始。

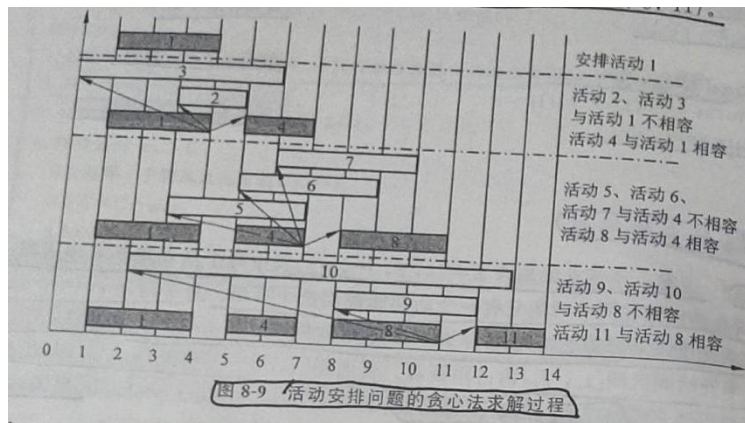
由于活动占用资源的时间没有限制，因此，后一种贪心选择更为合理。从直观上，按这种策略选择相容活动可以为未安排的活动留下尽可能多的时间，也就是说，这种贪心选择的目的是使剩余时间段极大化，以便安排尽可能多的相容活动。

为了在每一次贪心选择时都能快速查找具有最早结束时间的相容活动，可以将 n 个活动按结束时间非降序排列。这样，在贪心选择时取当前活动集合中结束时间最早的活动就归结为取当前活动集合中排在最前面的活动。例如有 11 个活动等待安排，这些活动按结束时间的非降序排列如表 8-1 所示。

表 8-1 11 个活动的开始时间和结束时间

序号	开始	结束	序号	开始	结束	序号	开始	结束
i	s_i	f_i	i	s_i	f_i	i	s_i	f_i
1	1	4	5	3	8	9	8	12
2	3	5	6	5	9	10	2	13
3	0	6	7	6	10	11	12	14
4	5	7	8	8	11			

贪心法求解活动安排问题的过程如图 8-9 所示，其中阴影长条表示该活动已加入解集合中，空白长条表示该活动是当前正在检查相容性的活动。首先选择活动 1 加入解集合，因为活动 1 具有最早结束时间；活动 2 和活动 3 与活动 1 不相容，所以舍弃它们；活动 4 与活动 1 相容，因此将活动 4 加入解集合；然后在剩下的活动中找与活动 4 相容并具有最早结束时间的活动，依次类推。最终被选定的活动集合为 {1,4,8,11}。



【算法】 设有 n 个活动等待安排， s_i 表示活动 i 的起始时间， f_i 表示活动 i 的结束时间($1 \leq i \leq n$)，集合 B 存放选定的活动，算法如下。

算法：活动安排问题 ActiveManage

输入： n 个活动的开始时间 $\{s_1, s_2, \dots, s_n\}$ 和结束时间 $\{f_1, f_2, \dots, f_n\}$

输出： 选定的活动集合 B

1. 对 $\{f_1, f_2, \dots, f_n\}$ 按非降序排序，同时相应地调整 $\{s_1, s_2, \dots, s_n\}$;

2. 最优解中包含活动 1: $B = \{1\}$; $j = 1$;

3. 循环变量 i 从 2 至 n 依次考查每一个活动:

3.1 如果($s_i \geq f_j$), 则 $B=B+\{j\}$; $j=i$;

3.2 $i++$;

【算法分析】步骤 1 将活动按结束时间从小到大排序, 时间代价是 $O(n \log 2n)$, 步骤 3 依次考查每一个活动, 时间代价是 $O(n)$, 因此, 算法的时间复杂度为 $O(n \log 2n)$ 。

【算法实现】简单起见, 假设数组 $s[n]$ 和 $f[n]$ 已按结束时间非降序排列, 数组 $B[n]$ 存储安排的活动, 若活动 i 可以安排, 则 $B[i]=1$, 程序如下。

```
int ActiveManage(int s[],int f[],int B[],int n)
{
    int i, j, count;
    B[0]=1; j=0; count=1;
    for (i=1; i < n; i++)
    {
        if (s[i] >= f[j]) //活动 i 与活动 j 相容
        {
            B[i]=1; count++; //安排活动
            j=i; //j 是目前安排的最后一个活动
        }
        else B[i]=0;
    }
    return count; //返回已安排的活动个数
```


动态规划法：

动态规划用来求解多阶段决策最优化问题。

多阶段决策过程：

【问题】该问题有 n 个输入，问题的解由这 n 个输入的一个子集组成，这个子集必须满足某些事先给定的约束条件，满足约束条件的解称为问题的可行解。满足约束条件的可行解可能不止一个，为了衡量这些可行解的优劣，通常以函数的形式给出一定的评价标准，这些标准函数称为目标函数，也称评价函数。目标函数的极值(极大或极小)称为最优值，使目标函数取得极值的可行解称为最优解，这类问题称为最优化问题。

例如，在 0/1 背包问题中，物品 i 或者被装入背包，或者不被装入背包，设 x_i 表示物品 i 装入背包的情况，则当 $x_i=0$ 时，表示物品 i 没有被装入背包， $x_i=1$ 时，表示物品 i 被装入背包。根据问题的描述，有如下约束条件和目标函数：

$$\begin{cases} \sum_{i=1}^n w_i x_i \leq C \\ x_i \in \{0, 1\} \quad 1 \leq i \leq n \end{cases} \quad (9-1)$$

$$\max \sum_{i=1}^n v_i x_i$$

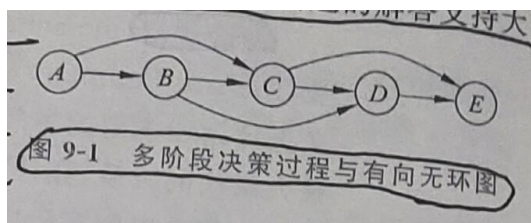
归结为寻找满足式(9-1)的约束条件，并使式(9-2)的目标函数达到最大。

0/1 背包问题归结为寻找满足式(9-1)的约束条件，并使式(9-2)的目标函数达到最大的解向量 $X=(x_1, x_2, \dots, x_n)$ 。

最优化问题的求解过程往往可以划分为若干个阶段，每一阶段的决策都依赖于前一阶段的状态，由决策采取的操作使状态发生转移，成为下一阶段决策的依据。如果对一个状态可以作出多个决策，每一个

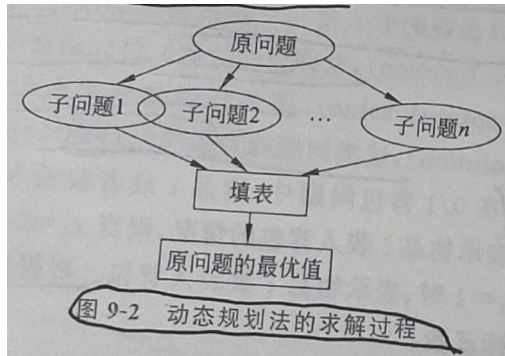
决策都可以产生一个新的状态，则动态规划的决策过程可以抽象为一个图模型，如图 9-1 所示，其中，顶点表示状态，有向边 $\langle v_i, v_j \rangle$ 表示某个决策将状态 v_i 转换为状态 v_j ，并且图模型为有向无环图。

在多阶段决策过程中，由于每一阶段的决策都与前一阶段的状态有关，因此，可以把每一阶段都作为一个子问题来处理，然后按照由小问题到大问题，以小问题的解答支持大问题求解的模式，依次求解所有子问题，最终得到原问题的解。例如，对于图 9-1 所示的决策过程，原问题 E 的解依赖于子问题 C 和 D 的解，子问题 D 的解依赖于子问题 C 和 B 的解，子问题 C 的解依赖于子问题 A 和 B 的解，子问题 B 的解依赖于子问题 A 的解，因此，动态规划的求解过程从初始子问题 A 开始，逐步求解并记录各子问题的解，直至得到原问题 E 的解。



动态规划法的设计思想：

动态规划法将待求解问题分解成若干个相互重叠的子问题，每个子问题都对应决策过程的一个阶段，一般来说，子问题的重叠关系表现在对给定问题求解的递推关系(称为动态规划函数)中，将子问题的最优值求解一次并填入表中，当需要再次求解该子问题时，可以通过查表获得该子问题的最优值，从而避免了大量重复计算。一般来说，动态规划法的求解过程由以下三个阶段组成，如图 9-2 所示。



(1)划分子问题：将原问题的求解过程划分为若干个阶段，每个阶段对应一个子问题，并且子问题之间具有重叠关系。

(2)动态规划函数：根据子问题之间的重叠关系找到子问题满足的递推关系式，这是动态规划法的关键。

(3)填写表格：根据动态规划函数设计表格，以自底向上的方式计算各个子问题的最优值并填表，实现动态规划过程。

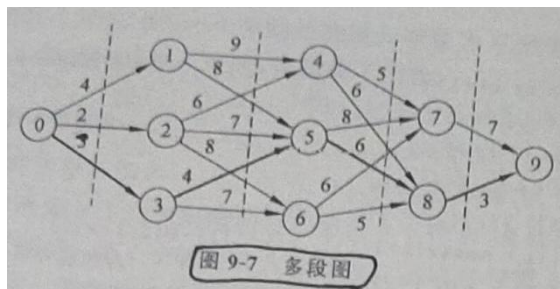
图问题中的动态规划法

多段图的最短路径：

【问题】 设图 $G=(V,E)$ 是一个带权有向图，顶点集合 V 划分成 k 个互不相交的子集 $V_i (2 \leq k \leq n, 1 \leq i \leq k)$ ，使得 E 中任何一条边 (u,v) ，必有 u 属于 V_i, v 属于 $V_{i+m} (1 \leq i < k, 1 < i+m \leq k)$ ，则称图 G 为多段图，称 s 属于 V_1 为源点， t 属于 V_k 为终点，多段图的最短路径问题求从源点到终点的最小代价路径。

【想法】 由于多段图将顶点划分为 k 个互不相交的子集，可以将多段图划分为 k 段，每一段都包含顶点的一个子集，根据多段图的定

义，每个子集中的顶点都互不邻接。将多段图的顶点按照段的顺序进行编号，同一段内顶点的顺序无关紧要。假设图的顶点个数为 n ，则源点 s 的编号为 0 ，终点 t 的编号为 $n-1$ ，并且对图中的任何一条边 $\langle u, v \rangle$ ，顶点 u 的编号小于顶点 v 的编号。图 9-7 所示是一个含有 10 个顶点的多段图。



设 $d(s, t)$ 表示从源点 s 到终点 t 的最短路径长度， c_{uv} 表示有向边 $\langle u, v \rangle$ 上的权值。考虑初始子问题 $d(s, s)$ ，路径尚未出发，显然 $d(s, s) = 0$ 。

考虑重叠子问题，设 $d(s, v)$ 表示源点 s 到顶点 v 的最短路径长度，显然有下式成立： $d(s, v) = \min\{d(s, u) + c_{uv}\}$ ， $\langle u, v \rangle$ 属于 E (9-12)

// $s \dots u v$ 先求 s 到 u 的最短路径长度，再加上 $\langle u, v \rangle$ 边上的权值

为了求得相应的最短路径，设 $p(s, t)$ 记载求解过程中的决策，其中 $p(s, v)$ 表示在计算 $d(s, v)$ 时的决策，即满足式(9-12)的顶点 u 。对于图 9-7 所示多段图，动态规划法的填表如表 9-1 所示，具体过程如下。

首先求解初始子问题，直接得： $d(0, 0) = 0, p(0, 0) = -1$ 。

再求解第一个阶段的子问题，有

$$d(0, 1) = d(0, 0) + c_{01} = 4, p(0, 1) = 0$$

$$d(0, 2) = d(0, 0) + c_{02} = 2, p(0, 2) = 0$$

$$d(0, 3) = d(0, 0) + c_{03} = 3, p(0, 3) = 0$$

再求解第二个阶段的子问题，有

$$d(0,4)=\min\{d(0,1)+c_{14},d(0,2)+c_{24}\}=\min(4+9,2+6)=8, p(0,4)=2$$

$$d(0,5)=\min\{d(0,1)+c_{15},d(0,2)+c_{25},d(0,3)+c_{35}\}=\min(4+8,2+7,3+4)=7,$$

$$p(0,5)=3$$

$$d(0,6)=\min\{d(0,2)+c_{26},d(0,3)+c_{36}\}=\min(2+8,3+7)=10, p(0,6)=2$$

再求解第三个阶段的子问题，有

$$d(0,7)=\min\{d(0,4)+c_{47},d(0,5)+c_{57},d(0,6)+c_{67}\}=\min(8+5,7+8,10+6)=13,$$

$$p(0,7)=4$$

$$d(0,8)=\min\{d(0,4)+c_{48},d(0,5)+c_{58},d(0,6)+c_{68}\}=\min(8+6,7+6,10+5)=13,$$

$$p(0,8)=5$$

直至最后一个阶段的子问题，有

$$d(0,9)=\min\{d(0,7)+c_{79},d(0,8)+c_{89}\}=\min\{13+7,13+3\}=16, p(0,9)=8$$

得到最短路径长度为 16。从 $p(0,9)$ 开始进行回溯， $p(0,9)=8$ ，回溯到 $p(0,8)$ ； $p(0,8)=5$ ，回溯到 $p(0,5)$ ； $p(0,5)=3$ ，回溯到 $p(0,3)$ ； $p(0,3)=0$ ，得到最短路径为 $0 \rightarrow 3 \rightarrow 5 \rightarrow 8 \rightarrow 9$ 。

表 9-1 多段图最短路径问题的填表过程

下标	$d(s, v)$	$p(s, v)$	下标	$d(s, v)$	$p(s, v)$
0	0	-1	5	7	3
1	4	0	6	10	2
2	2	0	7	13	4
3	3	0	8	13	5
4	8	2	9	16	8

最短路径是从 0 到 4 经过 2，最短代价为 8。
多段图采用代价矩阵 $arc[n][n]$ 存储，数组 $cost[n]$ 存储最短路径长度。

【算法实现】多段图采用代价矩阵 $\text{arc}[n][n]$ 存储，数组 $\text{cost}[n]$ 存储最短路径长度， $\text{cost}[j]$ 表示从源点 s 到顶点 j 的最短路径长度，数组 $\text{path}[n]$ 记录状态转移， $\text{path}[j]$ 表示从源点 i 到顶点 j 的路径上顶点 j 的前一个顶点，程序如下。

```
int ShortestPath(int arc[100][100], int n)
{
    int i, j, cost[n], path[n];

    cost[0]=0; path[0]=-1;    //顶点 0 为源点
    for(j=1; j<n; j++)        //执行填表工作
    {
        cost[j]=1000; //假定权值最大不超过 1000
        for(i=0; i<j; i++)    //考查所有入边
            //设置 i 找 j 之前的边。
            if (cost[i]+arc[i][j]<cost[j])    //对应 9-12
            {
                cost[j]=cost[i]+arc[i][j];    //例如找从 0 到 4 的最短代价：找到 4
                                                的前一个顶点即 2(最短)，然后从 0 到 2
                                                的最短路径长度加上 2 与 4 之间的代价。
                path[j]=i;
            }
    }
}
```

```

cout<<--n;          //输出终点
for(i=n; path[i] >=0;)    //依次输出 path[i] 回溯。
{
    cout<<"<-"<<path[i]; //引号包围的是箭头。
    i=path[i];    //求得路径上顶点 i 的前一个顶点
}
return cost[n-1];    //返回最短路径长度
}

```

【算法分析】第一个循环是两层嵌套 for 循环依次计算从源点到各个顶点的最短路径长度，执行次数为 $O(n^2)$ 。第二个循环输出最短路径经过的顶点，设多段图划分为 k 段，其时间性能是 $O(k)$ 。所以，算法的时间复杂度为 $O(n^2)$ 。

TSP 问题：是指旅行家要旅行 n 个城市，要求各个城市经历且仅经历一次，然后回到出发城市，并要求所走的路程最短。

【想法】假设从顶点 i 出发，令 $V'=V-i$ ，设 $d(i,V')$ 表示从顶点 i 出发经过 V' 中各个顶点一次且仅一次，最后回到出发点 i 的最短路径长度。设 c_{uv} 表示边 $\langle u,v \rangle$ 上的权值，考虑初始子问题，回到出发点 i 之前只经过一个顶点，设 $d(k,\{ \})$ 表示从顶点 k 回到顶点 i ，显然有：

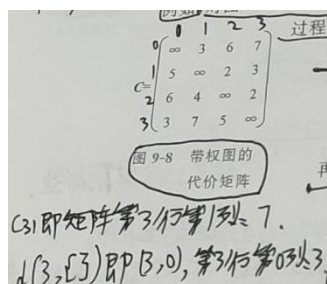
$$d(k,\{ \})=c_{ki} \quad k \text{ 属于 } V \text{ 并且 } k \neq i \quad (9-13)$$

考虑重叠子问题，设 $d(k,V'-\{k\})$ 表示从顶点 k 出发经过 $V'-\{k\}$ 中各个顶点一次且仅一次，最后回到出发点 i 的最短路径长度，则：

$$d(k,V'-\{k\})=\min\{c_{kj}+d(j,V'-\{j\})\} \quad (k \text{ 属于 } V', \langle k,j \rangle \text{ 属于 } E, j \text{ 属于 } V'-\{k\}) \quad (9-14)$$

注意： j 是经过的一个顶点。

例如，对图 9-8 所示带权图的代价矩阵，动态规划法的填表过程如图 9-9 所示，具体过程如下(括号中是该决策引起的状态转移)。



首先计算初始子问题，可以直接获得：

$$d(1,\{\})=c_{10}=5(1\rightarrow 0)$$

$$d(2,\{\})=c_{20}=6(2\rightarrow 0)$$

$$d(3,\{\})=c_{30}=3(3\rightarrow 0) \quad //d(3,\{\})\text{即}(3,0), \text{第}3\text{行第}0\text{列}: 3$$

再求解第一个阶段的子问题，有

$$d(1,\{2\})=c_{12}+d(2,\{\})=2+6=8(1\rightarrow 2)$$

$$d(1,\{3\})=c_{13}+d(3,\{\})=3+3=6(1\rightarrow 3)$$

$$d(2,\{1\})=c_{21}+d(1,\{\})=4+5=9(2\rightarrow 1)$$

$$d(2,\{3\})=c_{23}+d(3,\{\})=2+3=5(2\rightarrow 3)$$

$$d(3,\{1\})=c_{31}+d(1,\{\})=7+5=12(3\rightarrow 1)$$

$$d(3,\{2\})=c_{32}+d(2,\{\})=5+6=11(3\rightarrow 2)$$

再求解第二个阶段的子问题，有

$$\begin{aligned} d(1,\{2,3\}) &= \min(c_{12}+d(2,\{3\}), c_{13}+d(3,\{2\})) \\ &= \min\{2+5, 3+11\}=7(1\rightarrow 2) \end{aligned}$$

因为最终选择括号前半部分 c_{12} ，所以 $(1\rightarrow 2)$

$$\begin{aligned} d(2,\{1,3\}) &= \min(c_{21}+d(1,\{3\}), c_{23}+d(3,\{1\})) \\ &= \min\{4+6, 2+12\}=10(2\rightarrow 1) \end{aligned}$$

因为 c_{21} ，所以 $(2\rightarrow 1)$

$$\begin{aligned} d(3,\{1,2\}) &= \min(c_{31}+d(1,\{2\}), c_{32}+d(2,\{1\})) \\ &= \min(7+8, 5+9)=14(3\rightarrow 2) \end{aligned}$$

因为 c_{32} ，所以 $(3\rightarrow 2)$

直至最后一个阶段的子问题，有

$$d(0,\{1,2,3\})=\min(c_{01}+d(1,\{2,3\}),c_{02}+d(2,\{1,3\}),c_{03}+d(3,\{1,2\}))$$

$$=\min(3+7,6+10,7+14)=10(0\rightarrow1)$$

得到 TSP 问题从顶点 0 出发的最短路径长度为 10，最短路径是 0→1→2→3→0。(找出每个阶段子问题的最短路径)

以上计算过程例如： $d(2,\{1,3\})=\min(c_{21}+d(1,\{3\}),c_{23}+d(3,\{1\}))$

即： $c_{21}+c_{13}+d(3,\{ \}),c_{23}+c_{31}+d(1,\{ \})$

即： $(4+3+3,2+7+5)=\min\{4+6,2+12\}=10(2\rightarrow1)$

动态规划阶段		初始	第一个阶段			第二个阶段			第三个阶段
经过顶点集合		()	(1)	(2)	(3)	(1, 2)	(1, 3)	(2, 3)	(1, 2, 3)
出发顶点	0								10
	1	5		8	6			7	
	2	6	9		5		10		
	3	3	12	11		14			

图 9-9 动态规划法的填表过程

【算法】 设图 G 采用代价矩阵 $\text{arc}[n][n]$ 存储，简单起见，假定从顶点 0 出发求解 TSP 问题。首先，按个数为 1、2、...、 $n-1$ 的顺序生成 $1\sim n-1$ 个元素的子集存放在数组 $V[2^n-1]$ 中，例如当 $n=4$ 时， $V[1]=\{1\}, V[2]=\{2\}, V[3]=\{3\}, V[4]=\{1,2\}, V[5]=\{1,3\}, V[6]=\{2,3\}, V[7]=\{1,2,3\}$ 。设数组 $d[n][2^n-1]$ 存放迭代结果，其中 $d[i][j]$ 表示从顶点 i 经过子集 $V[j]$ 中的顶点一次且仅一次，最后回到出发点 0 的最短路径长度，算法如下。

算法：TSP 问题

输入：图的代价矩阵 $\text{arc}[n][n]$

输出：从顶点 0 出发 TSP 问题的最短路径长度

1.初始化第 0 列：

```
for(i=1;i<n;i++)
```

```
    d[i][0]=arc[i][0];
```

2.依次处理每一个子集数组 $V[2^n-1]$

```
for(i=1;i<n;i++)
```

```
    if(子集  $V[j]$  中不包含 i)
```

```
        对  $V[j]$  中的每个元素 k, 计算  $d[i][j]=\min\{arc[i][k]+d[k][j-1]\};$ 
```

3.输出最短路径长度 $d[0][2^{n-1}-1];$

//即上例中的 $V[7]$ 。

【算法分析】 算法需要对顶点集合 $\{1,2,\dots,n-1\}$ 的每一个子集都进行操作，因此时间复杂度为 $\Omega(2^n)$ 。和蛮力法相比，动态规划法求解 TSP 问题，把原来的时间复杂度是 $\Omega(n!)$ 的排列问题转化为组合问题，从而降低了算法的时间复杂度，但仍需要指数时间。